

软件设计规格说明书

项目名称：WeaveDoc

小组名称及成员：TreeNewBee

课程名称 软件工程

提交日期 2026 年 5 月 24 日

版本号 v1.0.0

摘要

WeaveDoc 是一款面向学术场景的全离线智能写作工具，旨在解决学术文档编辑、格式转换与文献问答中的核心痛点。系统采用分层架构设计，集成 Markdown 实时编辑、PDF 双屏联动、AFD 模板驱动的高保真格式导出、离线 RAG 文献问答等核心功能。通过本地 GGUF 模型推理、Pandoc 文档转换管线与 WebView2 跨端渲染技术，实现无需联网的完整学术写作闭环。本设计说明书详细阐述了系统的架构选型、界面原型、核心用例执行流程、关键类职责与数据存储策略，并通过 UML 模型完整呈现系统静态结构与动态行为，为开发实现提供全面指导。

目录

1. 引言

- 编写目的
- 项目背景
- 参考资料
- 术语定义
- 小组分工

2. 系统架构设计

- 需求回顾
- 架构选型分析
- 系统总体结构
- 接口设计
- 关键技术与设计决策
- 系统架构 UML 模型

3. 界面设计

- 用户分析
- 界面风格与规范
- 界面原型设计
- 界面交互设计

5. 界面设计 UML 模型
4. 详细设计
 1. 用例详细设计
 2. 类设计
 3. 数据设计
 4. 详细设计 UML 模型
5. 设计总结
 1. 设计成果概述
 2. 设计优势与不足
 3. 后续工作展望
6. 参考文献
7. 附录
 1. 系统架构设计评审表
 2. 界面设计评审表
 3. 详细设计评审表
 4. PlantUML 源代码
 5. UML 图 PNG 图片

1 引言

1.1 编写目的

本文档是 **WeaveDoc 学术写作智能工作站** 的软件设计说明书，旨在详细记录系统的架构设计、界面设计及核心模块的详细设计方案。本文档的预期读者包括：

- **开发团队**：作为编码实现和单元测试的直接依据；
- **项目评审人员**：评估设计方案的技术可行性、完整性与合理性；
- **后续维护者**：理解系统结构、接口约定和关键设计决策，便于持续迭代。

通过本文档，读者能够从宏观架构到微观类设计，全面理解 WeaveDoc 如何通过 Markdown 编辑、高保真学术排版导出、离线 RAG 文献问答及双屏联动等能力，解决高校研究者学术写作中的效率痛点。

1.2 项目背景

在高校与科研场景中，学术写作者普遍面临三大痛点：

1. **格式繁琐**：将 Markdown 等便捷写作格式转换为符合期刊或学位论文要求的 DOCX/PDF 时，样式映射易出错、手动调整耗时；

- 2. **文献利用低效**：本地存储的大量 PDF 文献难以与写作过程联动，基于内容的精准问答需依赖联网大模型，存在数据隐私与断网不可用风险；
- 3. **编辑与预览割裂**：传统编辑器缺乏实时预览和跨屏联动能力，用户需频繁切换窗口，写作体验碎片化。

WeaveDoc 基于 .NET 平台与 WebView2 混合技术，集成 Pandoc 文档转换管线、本地 GGUF 模型驱动的 RAG 问答引擎及 Monaco 编辑器，构建全离线、高保真、智能化的学术写作环境

1.3 参考资料

编号	资料名称	版本 / 说明
[1]	《软件工程》课程讲义	2025 版
[2]	IEEE 829-2008 软件设计文档标准	参考编写规范
[3]	WeaveDoc 需求规格说明书 V1.0	核心功能需求与用例定义
[4]	Pandoc 用户手册	Lua 筛选器与 DOCX 输出配置
[5]	Open XML SDK 文档	DOCX 样式编程接口
[6]	LLamaSharp 库文档	GGUF 模型加载与推理
[7]	Monaco Editor API 文档	编辑器二次开发接口
[8]	PDF.js 开发指南	内嵌 PDF 阅读器集成
[9]	WebView2 开发者文档	桌面应用混合渲染

1.4 术语定义

术语	定义
AFD	Academic Format Definition，学术格式定义。一套 JS 的样式模板，涵盖字体、字号、段间距、页眉页脚等论文要求。
RAG	Retrieval-Augmented Generation，检索增强生成。通过文献向量化检索，为 LLM 提供上下文，生成基于文献内容的方案。
GGUF	GPT-Generated Unified Format，一种本地大模型文件格式，用于存储量化后的 LLM 与 Embedding 模型。
Pandoc	通用文档转换器，WeaveDoc 使用其将 Markdown 转为 PDF。
WebView2	Microsoft 的嵌入式浏览器控件，用于承载 Monaco 编辑器及 PDF 查看器。
Lua Filter	Pandoc 的 Lua 脚本扩展，用于在转换过程中操纵 AST，定义样式映射。
OpenXML	DOCX 文件的底层 XML 操作 SDK，用于精细修正样式、页眉页脚。
Embedding	文本向量化技术，将文本映射为高维向量，用于语义相似度计算。
Reranker	重排序模型，对初步检索的候选文本块进行精细语义相关性排序，提升检索精度。

1.5 小组分工

成员姓名	架构设计职责	界面设计职责	详细设计
张启涵 (原质量与配置)	负责整体技术架构设计，包括系统模块划分、核心接口契约（如 <code>IDocumentConverter</code> 、 <code>IPdfConverter</code> ）定义、数据流转架构、技术选型评审与架构决策日志维护。	参与界面设计评审，协助界面设计文档的规范性审查，确保图标、字体等资源符合统一风格；对界面原型的可测试性提出建议。	负责自动测试用例（GitHub 质量静态详细设计设计文档性。
谭博友 (项目组长)	参与架构评审，协调子系统边界与集成方案，对架构设计提出优化意见。	主导界面设计： 负责主窗口布局、双屏联动区域、AI 助手侧边栏、导出配置窗口等全部 UI/UX 设计；使用 Figma 产出高保真原型，定义交互流程与视觉规范。	负责核心块（LLaMA 详细设计与序列图交付质量
王志钢 (编辑引擎开发)	负责编辑引擎子系统架构设计，包括 Markdown 编辑器与 PDF 阅读器的模块边界、通信机制（WebView2 互操作）及性能优化策略。	提供编辑器区域、PDF 控件及双屏联动交互的技术可行性建议，协助落地设计稿中的交互细节。	负责 Mor Markdown PDFium 渲染逻辑、及目录解计。
任逸青 (文档转换开发)	负责语义转换子系统架构设计，包括 AFD 协议解析器、Pandoc 转换管道及 OpenXML 样式映射的模块结构与扩展点设计。	参与界面设计评审，提供模板选择、BibTeX 导入等配置界面的业务需求输入，确保界面元素与后端能力匹配。	负责 AFD 用封装、板库、文计；编写计文档。

2 系统架构设计

2.1 需求回顾

2.1.1 核心功能需求

- 离线 AI 辅助写作：**支持基于本地大语言模型（LLM）的智能问答、内容生成与文献检索增强（RAG）。
- 学术文档编辑：**提供 Markdown 编辑器、PDF 预览、版本快照管理。
- 高保真格式导出：**支持基于 AFD 模板的学术文档导出（如 IEEE、WHU Thesis），兼容 GB/T 7714 等引用规范。

4. **文献管理**：与 Zotero 集成，支持 .bib 文件解析与引用映射。

2.1.2 非功能需求

- 1. **离线运行**：无需网络连接，所有计算与数据存储均在本地完成。
- 2. **性能**：模型加载时间 ≤ 10 秒（16GB RAM），文档导出时间 ≤ 5 秒（10 页以内）。
- 3. **兼容性**：支持 Windows 10/11（64 位）及 Ubuntu 22.04+/Fedora。

2.1.3 约束

- 1. **硬件依赖**：需 x86-64 CPU（支持 AVX2 指令集）、 ≥ 16 GB RAM、 ≥ 2 GB 磁盘空间。
- 2. **技术栈限制**：主程序基于 .NET 10.0 Runtime，AI 推理依赖 LlamaSharp（GGUF 模型）。

2.2 架构选型分析

2.2.1 备选架构模式对比

模式	优势	劣势
分层架构	层次清晰，便于分工开发；基础设施层可复用	跨层调用可能导致性能下降
微服务架构	服务独立部署，扩展性强	离线场景下进程间通信复杂

2.2.2 最终架构选择及理由

选择分层架构，理由如下：

- 1. **离线场景适配**：分层架构通过本地进程内调用（如 LlamaSharp）避免网络依赖，符合“无需网络连接”的约束。
- 2. **开发效率**：核心功能（如 AI 推理、文档转换）可封装为独立服务层，降低模块耦合度。
- 3. **资源控制**：通过 .NET Runtime 统一管理内存与线程，避免微服务的多进程资源竞争。

2.3 系统总体结构

2.3.1 子系统 / 模块划分

- 1. **表示层 (WeaveDoc.UI)**：负责用户交互（如 MainWindow、EditorView）。
- 2. **服务层**：
 - 1. **AI 子系统 (WeaveDoc.AI)**：提供 LLM 推理、向量检索（RAG）。

2. **转换子系统 (WeaveDoc.Conversion)**: 处理文档格式转换 (Markdown→PDF/DOCX)。

3. **文献子系统 (WeaveDoc.Literature)**: 管理文献数据 (Zotero 同步、BibTeX 解析)。

3. **基础设施层 (WeaveDoc.Infrastructure)**: 封装文件操作、数据库访问 (SQLite)、日志记录。

4. **核心实体层 (WeaveDoc.Core)**: 定义文档、模板、引用等数据模型。

2.3.2 子系统职责说明

子系统	职责
WeaveDoc.AI	加载 GGUF 模型，执行文本嵌入与生成；实现 RAG 流程（向量检索、生成）。
WeaveDoc.Conversion	解析 AFD 模板，调用 Pandoc CLI 完成格式转换；处理引用重排。
WeaveDoc.Literature	同步 Zotero 文献库，维护 .bib 文件与文档引用的映射关系。
WeaveDoc.Infrastructure	提供文件读写、SQLite 数据库操作、审计日志记录。

2.4 接口设计

2.4.1 子系统间接口

- IExportService**: 转换子系统向表示层暴露的导出接口（如 `ExportAsync(mdPath, templateId)`）。
- IRAGEngine**: AI 子系统向编辑子系统提供的检索增强接口（如 `SearchAsync(question, topK)`）。
- ICitationIndex**: 文献子系统向转换子系统提供的引用查询接口（如 `GetCitationMap(docId)`）。

2.4.2 外部接口

- Pandoc CLI**: 通过标准输入输出 (stdin/stdout) 与转换子系统通信，执行文档格式转换。
- Zotero**: 通过 Better BibTeX 插件导出 .bib 文件，文献子系统定期同步数据。
- SQLite**: 基础设施层通过 Microsoft.Data.Sqlite 访问本地数据库，存储向量索引与配置。

2.5 关键技术与设计决策

2.5.1 技术栈选择

技术	用途	
.NET 10.0	主程序运行时	跨平台支持 (Windows/Linux)，高性能
LlamaSharp	本地 LLM 推理	支持 GGUF 模型，无需 Python 环境依赖
Avalonia UI	跨平台 UI 框架	兼容 Windows 与 Linux，支持 MVVM
SQLite	本地数据存储	轻量级，支持向量索引 (通过 sqlite-v

2.5.2 重要设计决策记录

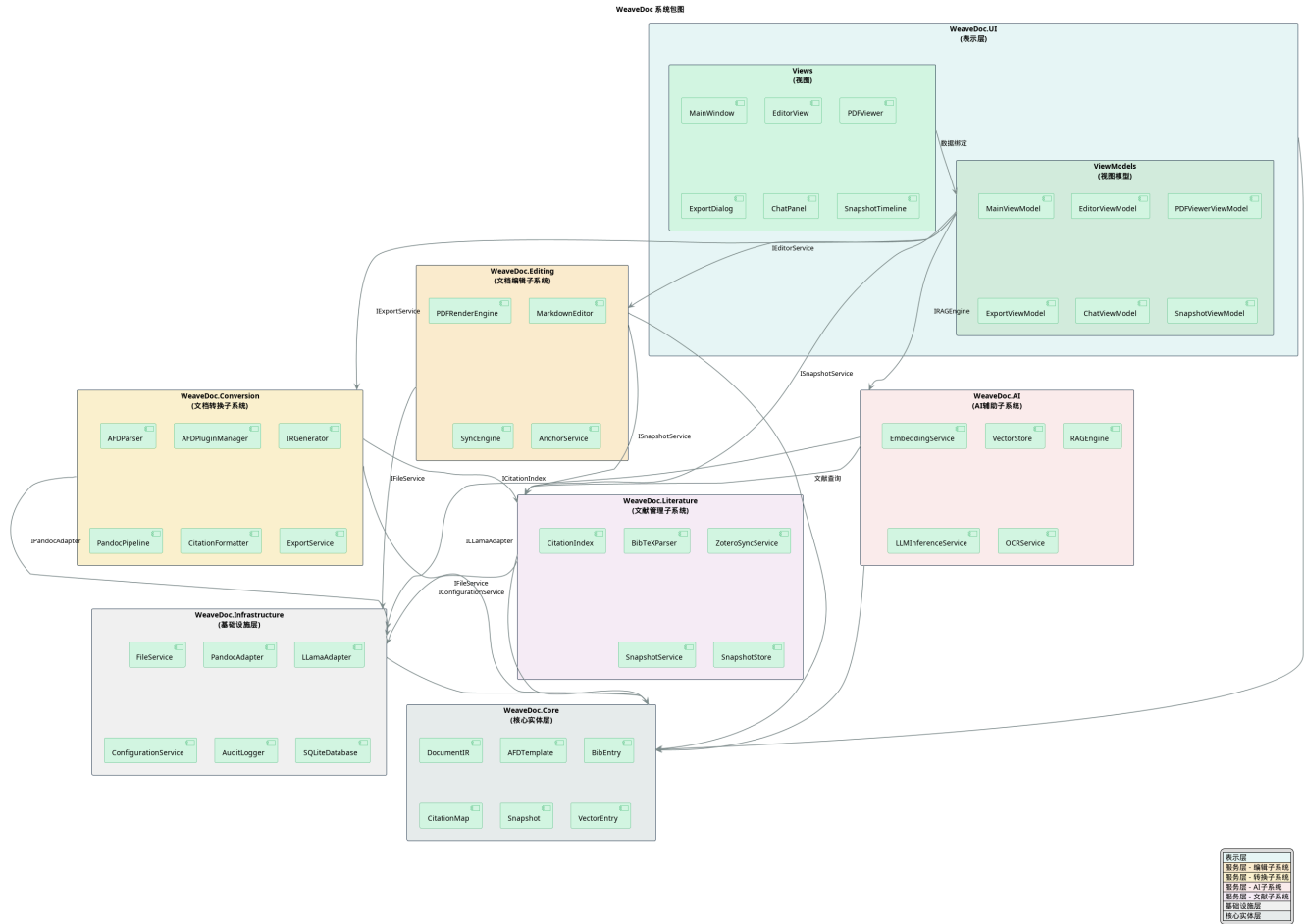
1. **进程内 AI 推理**：采用 LlamaSharp 而非独立 AI 服务，减少进程间通信开销，提升响应速度。
2. **模板驱动导出**：通过 AFD JSON 模板定义文档样式，实现格式与内容解耦，支持快速扩展新模板。
3. **离线优先**：所有模型文件 (GGUF)、文献数据 (.bib)、向量索引均存储于本地，确保无网络环境可用性。

2.5.3 技术风险分析及应对措施

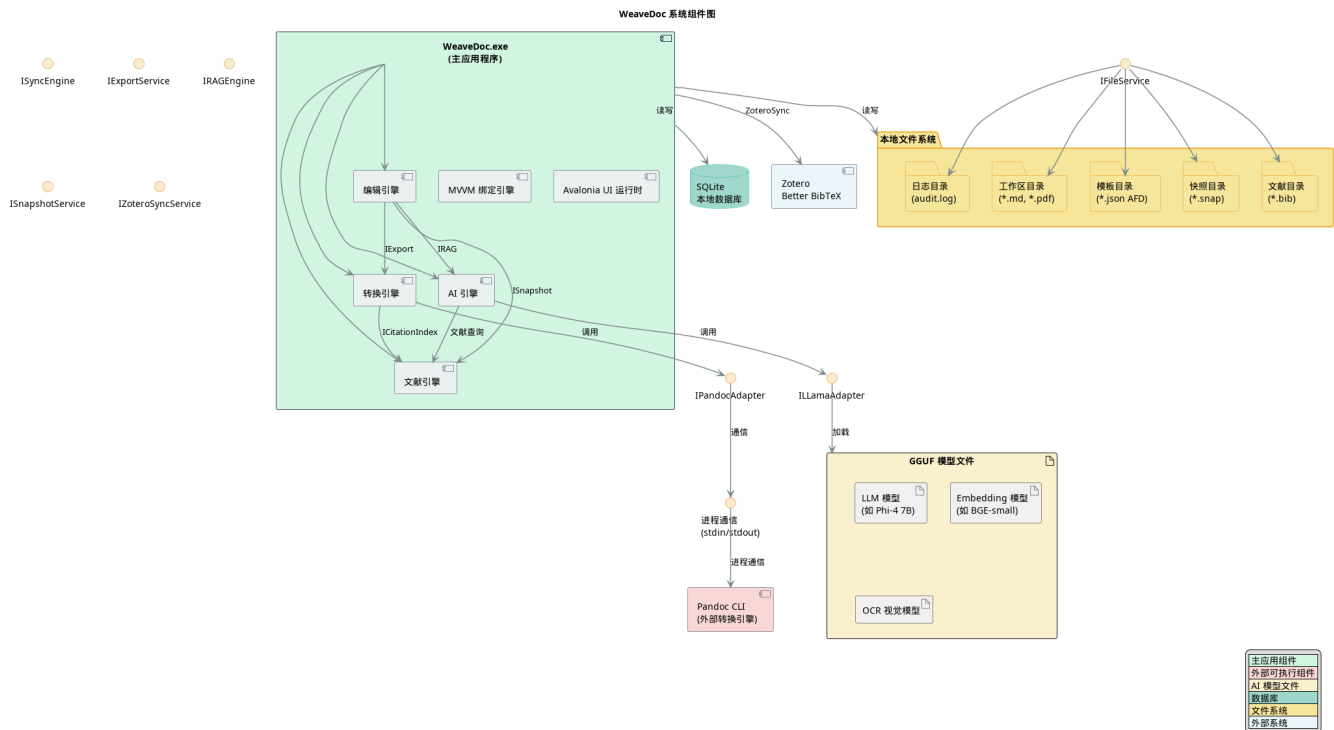
风险	影响	应对措施
模型加载内存溢出	程序崩溃	限制模型大小 ($\leq 4\text{GB}$)，启动时预检
Pandoc 转换兼容性	导出格式错误	内置 Pandoc 版本锁定 (v3.1.11)，提
Zotero 同步冲突	文献数据不一致	采用文件锁机制，同步前备份 .bib 文件

2.6 系统架构 UML 模型

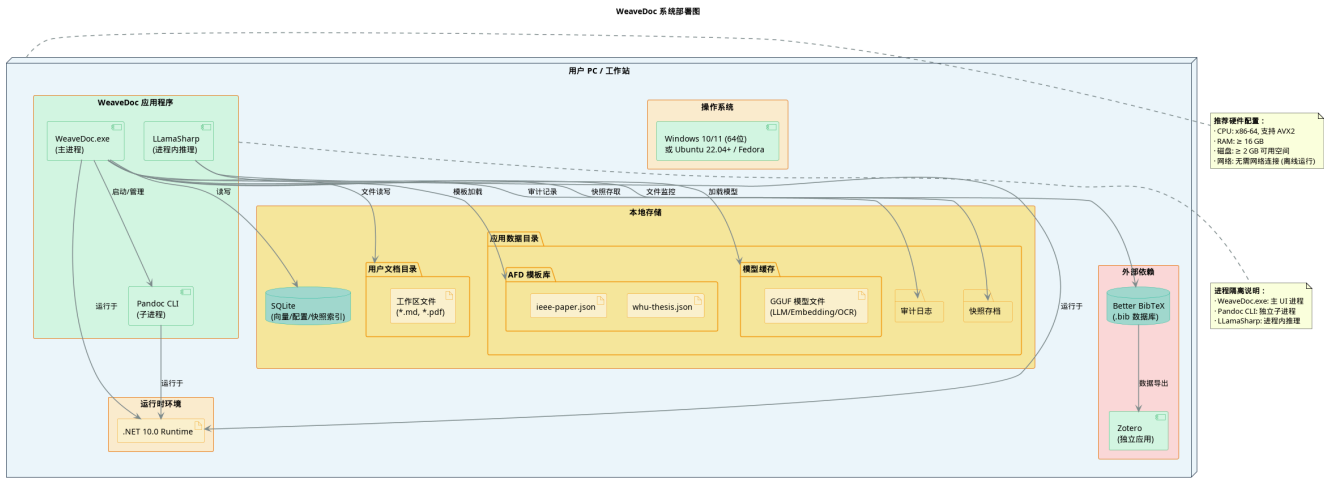
2.6.1 系统包图



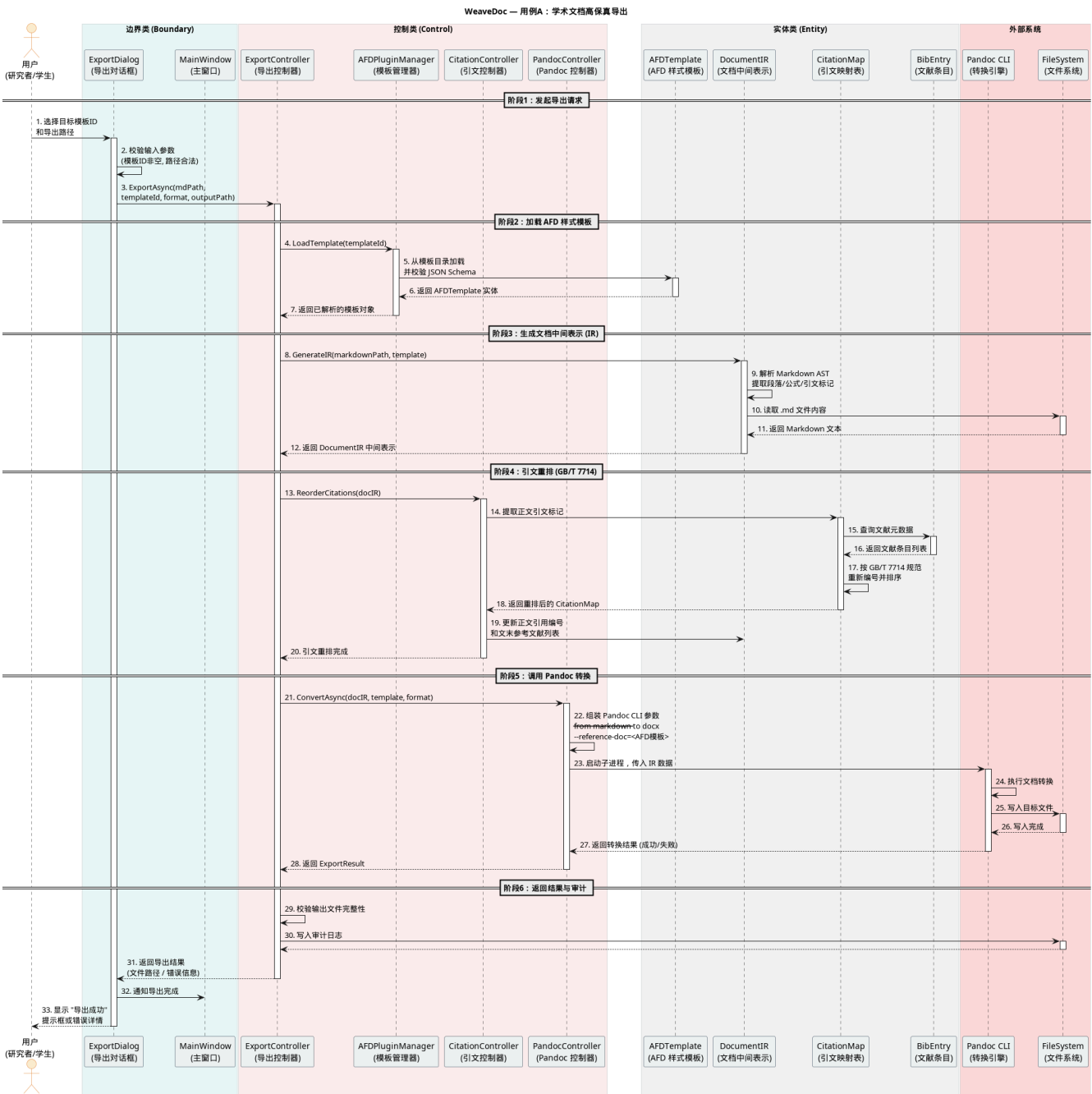
2.6.2 组件图

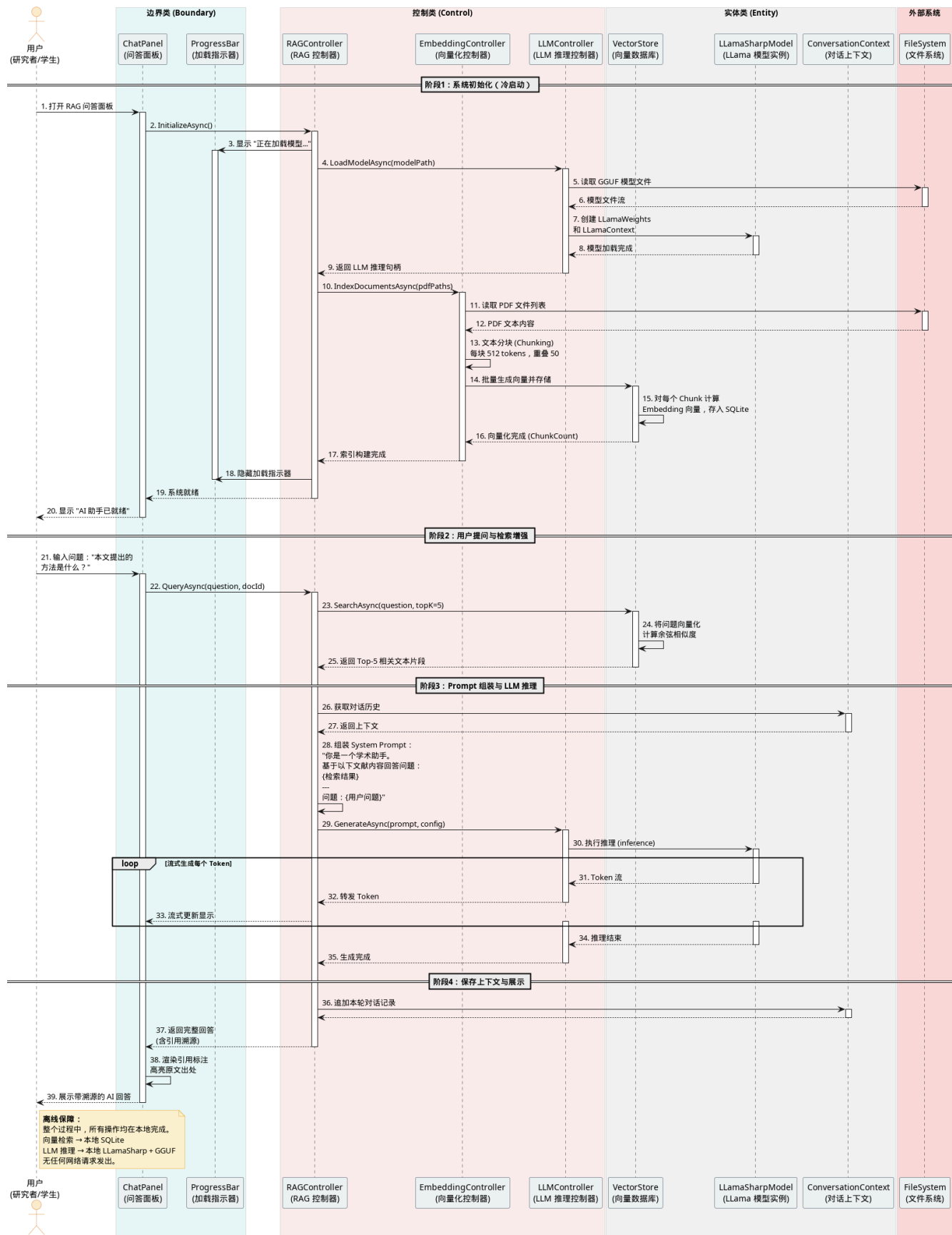


2.6.3 部署图



2.6.4 核心业务高层顺序图





3 界面设计

3.1 用户分析

WeaveDoc 面向学术论文写作与文献阅读场景，核心目标是将“文献阅读、Markdown 写作、参考文献管理、格式规范导出、离线 AI 辅助”整合到同一个本地学术工作台。用户在使用过程中既是论文内容的生产者，也是文献资料的检索者和整理者，因此系统需要同时满足高效写作、精准引用、隐私保护和低学习成本等需求。

3.1.1 目标用户特征

WeaveDoc 的主要目标用户为高校本科生、研究生及初级科研人员，尤其是需要频繁撰写课程论文、学位论文、实验报告和科研论文的理工科用户。该类用户通常具备一定的信息化工具使用经验，但不一定具备复杂排版系统或编程工具的深度使用能力。

从使用角色看，目标用户可归纳为两类任务角色：

用户角色	典型特征	核心诉求
学术写作者	需要长期撰写 Markdown 手稿、管理公式、图表、参考文献和导出格式	专注论文内容，将复杂排版和引用一致性交给系统处理
文献阅读者	需要阅读大量 PDF 文献，从原文中定位观点、公式、段落和图表	快速理解文献内容，并能将原文位置与写作内容建立关联

两类角色并非完全独立，而是同一用户在不同学术工作阶段的表现：在文献调研阶段，用户更关注 PDF 阅读、文献问答和公式提取；在论文写作阶段，用户更关注 Markdown 编辑、引文管理、格式导出和版本恢复。

目标用户具有以下特征：

学术任务密集：用户需要在较长周期内完成论文选题、文献调研、正文撰写、公式整理、引用标注和格式提交等任务。

已有本地工具习惯：多数用户已习惯使用 Zotero、Markdown、本地 PDF 阅读器等工具组合，能够接受基于文件夹和本地工作区的工作方式。

排版能力有限但规范要求高：用户通常熟悉基础 Markdown 语法，但普遍认为 LaTeX 学习门槛较高；同时，高校或期刊对页眉页脚、目录、公式、参考文献格式等要求严格。

对隐私和本地化敏感：论文草稿、未发表研究思路、PDF 文献和公式截图都具有较高隐私价值，用户需要尽量避免将内容上传至云端 AI 服务。

对结果可靠性敏感：引文编号错乱、参考文献缺失、公式乱码、导出失败等问题会直接影响论文提交质量，因此用户重视系统的校验、提示和可追溯能力。

重视响应速度：写作过程属于高频输入场景，键盘输入、语法高亮、引用预览、双屏联动等交互需要保持低延迟，避免打断思路。

存在一体化工具需求：需求资料显示，用户在 Word 模板处理、公式对齐和文献引用整理上存在明显耗时痛点，并对集成式学术工作台有较强需求。

3.1.2 用户使用习惯与需求

目标用户的使用习惯主要围绕本地文件、Markdown 写作、PDF 阅读和 Zotero 文献库展开。系统设计应尽量贴合这些既有习惯，在不显著增加学习成本的前提下，将分散工具整合为连续工作流。

用户使用习惯如下：

以本地文件夹作为工作入口：用户习惯按课程、课题或论文项目建立本地目录，保存 Markdown 文稿、PDF 文献、BibTeX 文件和导出结果。

以 Markdown 作为正文写作格式：用户倾向于使用轻量文本格式组织标题、段落、公式、表格和引用标记，希望保留源文件的可读性和可迁移性。

使用 Zotero 管理文献元数据：用户通过 Zotero 维护文献条目，并借助 Better BibTeX 导出 .bib 文件，用于论文引用和参考文献生成。

阅读与写作并行进行：用户在写作时需要频繁对照 PDF 原文，查找段落、图表和公式，并将原文位置插入到当前 Markdown 手稿中。

希望 AI 辅助但不希望依赖云端：用户希望系统能提供文献问答、摘要理解和公式 OCR 等智能能力，但核心数据处理应在本地完成。

导出阶段关注确定性结果：用户在提交前需要将 Markdown 手稿导出为符合学校或期刊规范的 Word/PDF 文件，且希望引文编号、文末参考文献和公式渲染保持一致。

需要版本安全感：论文写作周期长，用户可能反复修改章节内容，因此需要自动快照和可视化回滚能力，降低误改和丢失内容的风险。

用户核心需求如下：

需求类别	具体需求	对界面设计的要求
写作需求	Markdown 编辑、语法高亮、公式输入、引用标记预览	编辑器应占据核心工作区，输入反馈低延迟，保持沉浸式写作体验
阅读需求	PDF 浏览、缩放、翻页、段落定位、公式框选	PDF 面板需要与编辑器并列展示，支持稳定的双屏联动
引文需求	Zotero 同步、BibTeX 解析、引用键匹配、导出前校验	文献状态和引用问题需要在状态栏、文献面板和导出对话框中明确显示
导出需求	AFD 模板选择、Word/PDF 输出、Pandoc 转换、引文重排	导出流程应以步骤化对话框呈现，显示模板、路径、校验结果和阶段进度
AI 需求	离线 RAG 问答、带原文溯源回答、公式 OCR 转 LaTeX	AI 功能应作为辅助面板或浮层出现，不阻塞主编辑流程
隐私需求	文稿、PDF、OCR 和 AI 推理均在本地处理	不显示常驻联网依赖，联网动作应由用户显式触发
安全需求	自动保存、快照回滚、破坏性操作二次确认	快照恢复、删除模型、清空对话等操作必须提供确认和状态反馈
性能需求	打字响应、双屏跳转和 AI 首字响应可感知快速	高频交互路径应减少点击次数，耗时任务必须展示进度

综上，WeaveDoc 的界面不应采用轻量展示型或营销型布局，而应围绕“持续写作 + 密集阅读 + 精准管理”的学术工作方式组织界面，将高频操作置于固定区域，将辅助功能折叠到上下文面板或浮层中。

3.2 界面风格与规范

WeaveDoc 的界面设计应服务于本地学术工作台定位，强调信息密度、稳定结构、清晰状态和低干扰操作。界面不追求装饰性视觉效果，而是以文稿内容、PDF 原文、文献索引和系统状态为中心，尽量减少用户在工具之间切换和查找功能的成本。

3.2.1 整体风格定位

WeaveDoc 的整体风格定位为：**本地优先、隐私安全、高密度、IDE 风格的学术工作台。**

具体风格特征如下：

本地优先：系统围绕本地工作区、本地文档、本地模型和本地文献库展开。界面应弱化云端服务感，强调文件可控、路径明确和处理过程透明。

学术严谨：视觉语言保持克制、清晰和中性，避免过度装饰、强烈渐变和大面积营销式留白，突出论文写作工具应有的专业性。

高信息密度：用户需要同时查看 PDF、Markdown、文献列表、AI 回答和状态信息，因此界面采用紧凑间距、小字号、高密度列表和固定面板结构。

IDE 式工作流：主界面采用类似 VS Code 的分栏工作台结构，顶部保留菜单栏和工具栏，底部保留状态栏，中间通过可拖拽分屏承载 PDF 与 Markdown。

双核心视觉区：PDF 阅读区使用白色纸张式背景，强化“阅读原文”的稳定感；Markdown 编辑区使用深色代码编辑背景，强化“写作与结构化编辑”的沉浸感。

AI 辅助而非主导：AI 问答、公式 OCR 和向量化索引作为上下文辅助能力存在，不能遮挡或阻断主编辑区。AI 状态需要清晰可见，但不应抢占写作焦点。

状态显性化：AI 模型、内存占用、同步状态、引用数量、导出进度和错误信息都应在固定位置呈现，避免用户误以为系统卡死或数据丢失。

整体风格可参考以下工具组合：

VS Code：分栏编辑、状态栏、工具条、紧凑菜单和可拖拽面板。

Zotero：PDF 阅读、文献元数据和引用管理体验。

Obsidian：Markdown 源文件写作、链接化知识组织和本地文件优先体验。

3.2.2 设计规范（字体、色彩、布局、图标等）

1. 字体规范

界面字体应区分 UI 文字、代码编辑和 PDF 内容三类场景：

使用场景	字体规范	说明
UI 文字	Windows 使用 Segoe UI，Linux 使用 Noto Sans，后备为 system-ui, sans-serif	用于菜单、工具栏、面板标题、表单、对话框和状态栏
Markdown / 代码 / 引用键	JetBrains Mono，后备为 Fira Code、Cascadia Code、Consolas、monospace	用于 Markdown 编辑器、LaTeX 代码块、引用键和等宽状态信息
PDF 内容	使用 PDF 内嵌字体或文档自身字体	保持 PDF 原文排版，不强行替换字体

字号应采用紧凑阶梯，避免大字号占用工作区：

Token	字号	行高	字重	主要用途
Caption	10px	14px	400	状态栏、微型提示、OCR 置信度、时间戳
Body Dense	11px	16px	400	侧边栏列表、引用芯片、文件树、状态栏文本
Subheading	12px	16px	600	面板分组标题、对话框分区标题
Body Regular	13px	20px	400	核心 UI 字号，用于文献列表、聊天气泡、表单正文
Code Editor	14px	22px	400	Markdown 编辑器、LaTeX 代码块和源码内容
Panel Header	11px	12px	700	侧栏 chrome 标签和大写面板标题

2. 色彩规范

色彩系统以中性灰阶和学术蓝为主，避免高饱和装饰性色。亮色主题用于 PDF 阅读和一般界面表面，深色区域用于 Markdown 编辑器，形成“白色阅读 + 深色写作”的双核心视觉结构。

色彩类别	主要色值	使用场景
主背景	#FFFFFF	PDF 阅读器、主工作区阅读表面、模态表面
主文字	#1C2128	亮色主题正文和标题
编辑器背景	#0D1117	Markdown 编辑器深色编辑区
编辑器文字	#E6EDF3	Markdown 编辑器正文和代码文字
面板背景	#F8F9FA / #F0F6FC	侧边栏、上下文面板、柔和背景
边框分隔	#D8DEE4	分栏线、列表分隔线、输入框边界
主交互蓝	#0969DA	链接、激活文本、亮色主题主按钮和引用溯源
暗色交互蓝	#58A6FF	暗色主题链接、Tab 下划线、锚点链接

语义色用于明确表达状态，不应用作大面积装饰：

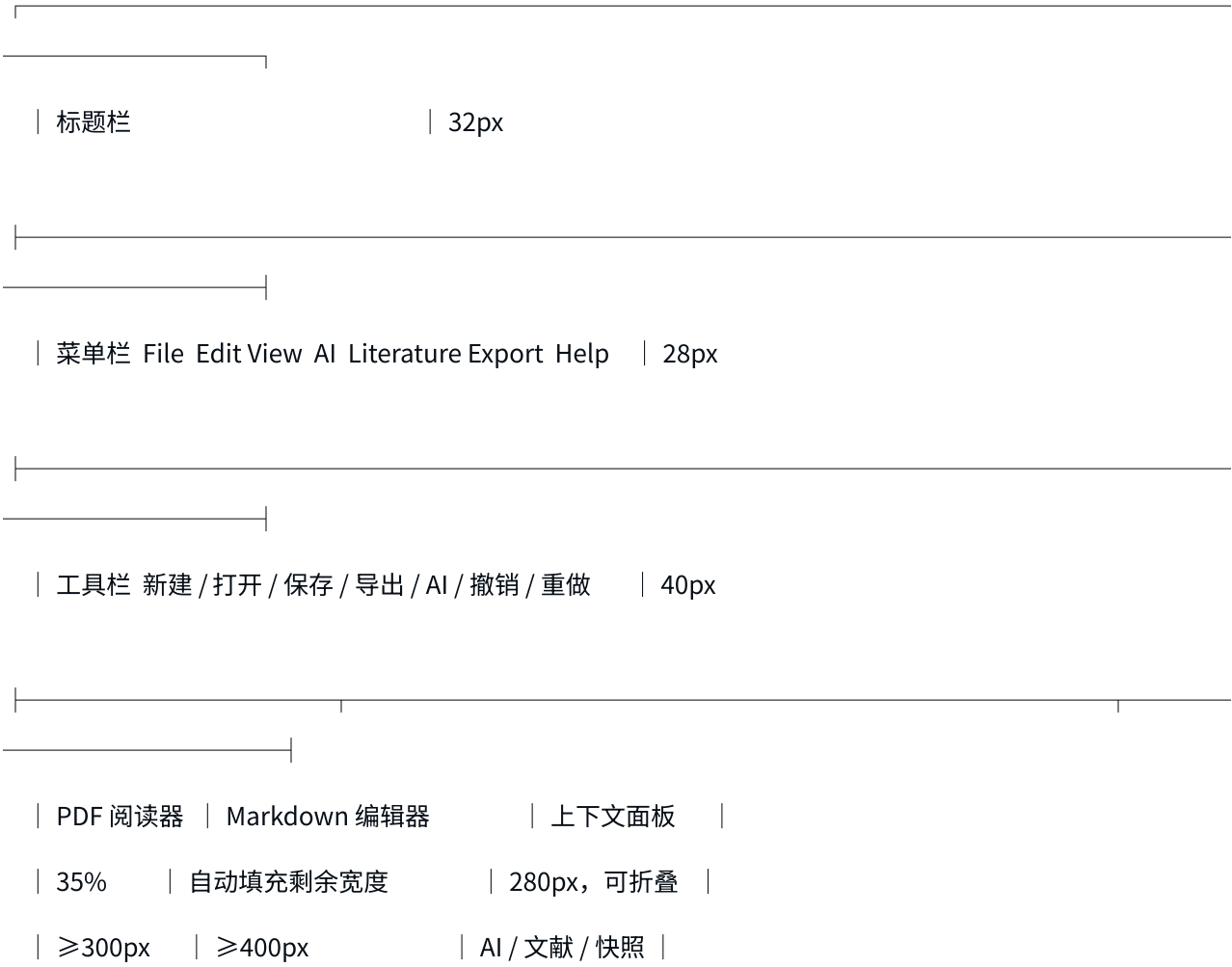
状态	色值	使用场景
成功	#1A7F37 / #3FB950	导出成功、AI 就绪、PDF 匹配成功
警告	#9A6700 / #D29922	引文警告、内存 70%-80%、部分识别
错误 / 破坏性	#CF222E / #F85149	导出失败、模型损坏、内存超阈值、删除或恢复确认
信息	#3B82F6	普通信息提示

AI 状态灯使用固定颜色，不随主题变化：

AI 状态	颜色	含义
unconfigured	#888888	未配置模型
unloaded	#F85149	已配置但未加载
loading	#D29922	模型加载中
idle	#3FB950	模型就绪
inference	#3FB950	模型推理中，使用呼吸动画

3. 布局规范

WeaveDoc 采用单窗口多面板结构，默认窗口尺寸为 1440 × 900px，最小窗口尺寸为 1024 × 700px。整体布局由标题栏、菜单栏、工具栏、主工作区和状态栏构成。





布局规则如下：

顶部标题栏高度为 32px，菜单栏高度为 28px，工具栏高度为 40px，状态栏高度为 24px。

主工作区采用三栏结构：左侧 PDF 阅读器、中间 Markdown 编辑器、右侧上下文面板。

PDF 阅读器默认占工作区宽度的 35%，最小宽度为 300px，可拖拽范围为 25%-50%。

Markdown 编辑器自动填充剩余空间，最小宽度为 400px。

上下文面板固定宽度为 280px，可折叠，承载 AI 问答、文献列表和快照时间轴。

窗口宽度大于等于 1440px 时默认展开上下文面板；1024-1439px 时默认折叠，用户可按需展开。

分隔线宽度为 2px，支持拖拽调整；双击分隔线重置为默认比例。

导出、设置、首次配置向导和帮助等任务使用模态窗口，避免主工作区结构被临时页面打散。

4. 间距规范

界面采用 4px 基准网格，所有 padding、margin 和 gap 应使用 4px 的倍数。常用间距如下：

Token	值	主要用途
space-2	2px	分隔线、图标与文字的紧贴间距
space-4	4px	紧凑元素间距、工具栏按钮内部图标 padding
space-8	8px	默认内边距、列表行垂直 padding、消息气泡间距
space-12	12px	面板内边距、表单字段间距、状态栏项间距
space-16	16px	模块间距、对话框内部区域间距
space-24	24px	对话框内容区到边缘、页面级分区间距
space-32	32px	大分区间距，原则上少用

具体应用要求如下：

工具栏按钮高度为 32px，图标居中显示。

单行输入框、下拉框和普通按钮高度为 32px。

AI 多行输入框高度为 60px，输入区整体高度为 88px。

文献列表行高度为 56px，模板列表行高度为 64px。

上下文面板内容区使用 12px 内边距，对话框内容区使用 24px 水平内边距和 16px 垂直分区间距。

5. 图标规范

图标应采用简洁线条风格，保持桌面生产力工具的克制感。

属性	规范
图标风格	Lucide / Feather 风格的简洁线性图标
标准尺寸	16 × 16px，用于工具栏、菜单和状态项
空状态尺寸	24 × 24px，用于空状态占位
描边宽度	1.5px
颜色	继承父级文字颜色，使用单色 CurrentColor
状态灯	8px 直径圆点，颜色按 AI 状态或语义状态映射

工具栏按钮优先使用图标，不直接显示文字；文字说明通过 tooltip 提供。页面原型中的符号类占位不应直接作为实现图标，应统一替换为 PathIcon、SvgImage 或 StatusDot。

6. 圆角与阴影规范

WeaveDoc 应保持桌面软件式锐利边界，避免 Web SaaS 风格的大圆角和柔和漂浮卡片。

Token	值	使用场景
radius-none	0px	外层窗口、菜单栏、状态栏、分屏分隔线
radius-sm	2px	工具栏按钮、复选框、引用芯片、输入框、行选择器
radius-md	4px	AI 气泡、下拉面板、公式 OCR 浮层、小型卡片
radius-lg	6px	模态窗口外壳，作为最大允许圆角

阴影层级应尽量克制：

主工作区、PDF 阅读器、Markdown 编辑器和上下文面板不使用阴影，依靠 1px 边框和 2px 分隔线区分区域。

菜单下拉、自动补全和状态弹层使用低层级阴影。

模态窗口和公式 OCR 浮动工具栏可使用中层级阴影，以表达其覆盖关系。

7. 状态与反馈规范

系统状态必须显性化，尤其是离线 AI、导出、引用校验和内存占用等高风险场景。

状态栏左侧显示光标位置、字数、引用数量、同步状态和未保存标记。

状态栏右侧显示 AI 模型状态和内存占用。

AI 面板顶部重复显示模型名称、加载状态、推理速度和内存比例，避免用户误判模型状态。

耗时超过 1 秒的操作必须显示明确进度，不应只显示无说明的无限加载动画。

导出失败时需要说明错误原因，并明确原始 Markdown 文稿不受影响。

恢复快照、删除模型、清空对话等破坏性操作必须二次确认；恢复快照前应生成保护性快照。

普通空状态应包含图标、主文案、辅助说明和至少一个操作入口；纯信息空状态可不提供操作按钮。

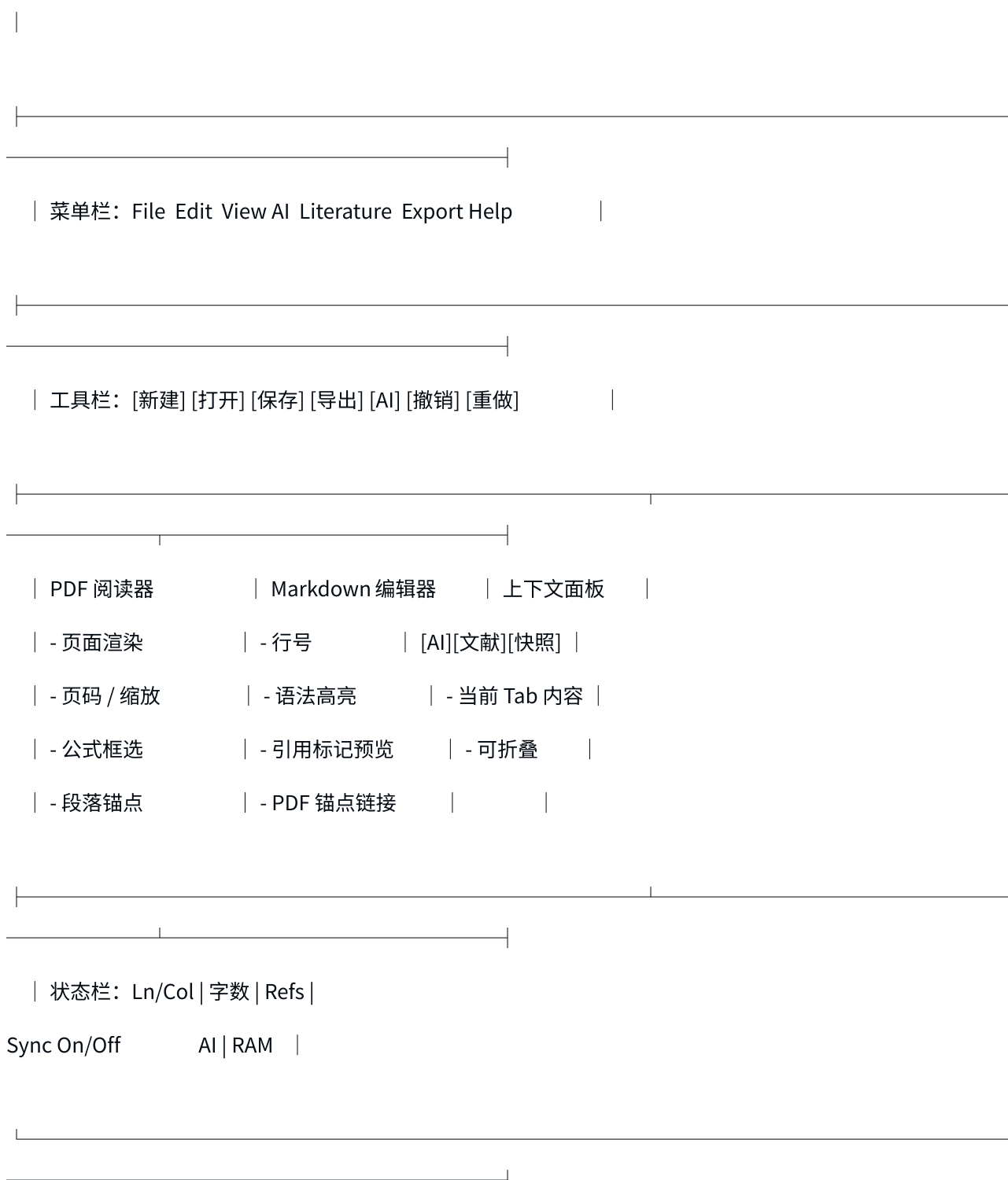
通过上述规范，WeaveDoc 的界面能够在保证高信息密度的同时，维持清晰的操作路径和稳定的学术工具气质，使用户在阅读、写作、引用、导出和 AI 辅助之间形成连续、高效、可信的工作流。

3.3 界面原型设计

WeaveDoc 的界面原型围绕“单窗口多面板”展开。系统启动后默认进入工作台，用户主要在同一窗口内完成 PDF 阅读、Markdown 写作、AI 问答、文献检索、快照回滚和导出操作。低频配置类任务通过模态窗口完成，临时上下文任务通过右侧面板或浮动工具栏完成。

3.3.1 主界面设计

主界面为 WeaveDoc 的常驻工作台页面，是用户停留时间最长、信息密度最高的界面。该界面采用固定顶部工具区、中央三栏工作区、底部状态栏的结构。



主界面模块如下：

模块	位置	原型内容	设计说明
标题栏	顶部	应用名称、工作区名称、当前文档名、窗口控制按钮	使用系统原生标题栏，突出当前工作上下文
菜单栏	标题栏下方	File、Edit、View、AI、Literature、Export、Help	提供完整功能入口，覆盖低频和高级操作
工具栏	菜单栏下方	新建、打开、保存、导出、AI、撤销、重做	高频操作图标化，导出按钮视觉权重最高
PDF 阅读器	工作区左侧	PDF 页面、页码、缩放、翻页、选区高亮、锚点标记	支持文献阅读、公式框选、锚点生成
Markdown 编辑器	工作区中部	文本编辑区、行号、语法高亮、引用 tooltip、锚点链接	作为核心写作区，保持深色沉浸式编辑体验
上下文面板	工作区右侧	AI 问答、文献列表、快照时间轴三类 Tab	承载辅助任务，可折叠以释放写作空间
状态栏	底部	光标、字数、引用数、同步状态、AI 状态、内存占用	提供系统运行状态和可点击跳转入口

主界面的默认布局规则如下：

默认窗口尺寸为 1440 × 900px，在该宽度下展开右侧上下文面板。

PDF 阅读器默认占剩余工作区宽度的 35%，Markdown 编辑器填充剩余宽度。

右侧上下文面板固定宽度为 280px，窗口宽度为 1024-1439px 时默认折叠。

PDF 与 Markdown 之间使用可拖拽分隔线，拖拽范围为 25%-50%，双击恢复默认比例。

工具栏按钮只显示图标，具体操作名称和快捷键通过 tooltip 提示。

3.3.2 功能模块界面设计

功能模块界面分为“上下文面板类”“模态窗口类”和“浮层类”三种形态。上下文面板适合写作过程中持续存在的辅助信息；模态窗口适合配置、导出等需要用户集中完成的原子任务；浮层适合与当前 PDF 区域强关联的即时操作。

功能模块	界面形态	入口	核心内容	关键状态
AI 问答面板	右侧上下文面板	工具栏 AI、View 菜单、Ctrl+Shift+A	模型状态条、对话消息列表、引用溯源链接、问题输入区	未配置、未加载、加载中、就绪、推理中、推理失败
文献列表面板	右侧上下文面板	View 菜单、Ctrl+Shift+L、状态栏 Refs	搜索过滤栏、统计摘要、文献条目、导入与同步操作	空态、正常列表、搜索无结果、同步中、同步失败、解析错误
快照时间轴面板	右侧上下文面板	View 菜单、Ctrl+Shift+T	快照统计、时间轴节点、只读预览、恢复按钮	空态、预览加载中、恢复确认、恢复执行中、快照损坏
导出对话框	模态窗口	工具栏导出、Export 菜单、Ctrl+Shift+E	模板选择、导出格式、输出路径、引文校验、导出进度	无模板、路径未选、校验成功、校验警告、结构错误、导出成功、导出失败
设置页面	模态窗口	File → Settings、Ctrl+,	通用设置、模型管理、Zotero 同步、模板库、快照策略	未配置、已配置、下载中、模型加载中、同步断连、磁盘不足
首次配置向导	多步骤模态	首次启动自动、Help → Setup Wizard	环境检测、模型配置、Zotero 关联、模板导入、完成摘要	检测通过、依赖缺失、配置跳过、下载失败、完成
公式提取浮层	PDF 上下文浮层	PDF 中框选公式区域	公式识别标题、LaTeX 代码块、复制、重新框选、关闭	识别中、识别成功、识别失败、模型未加载、部分识别
帮助 / 关于	模态窗口	Help 菜单、F1	版本信息、技术栈、快捷键、审计日志入口	正常展示、审计日志导出中、导出成功、导出失败

AI 问答面板原型

[AI 问答] [文献] [快照] [×]

● Phi-4 7B • 3.2 t/s • RAM 42%

用户：本文提出的方法是什么？

AI：本文提出了

[1] p.3 [2] p.7

[↓ 回到底部]

基于已加载的 PDF 文献提问

[输入你的问题] [发送]

AI 问答面板强调“状态先行、回答可溯源”。顶部模型状态条始终可见；回答以流式文本输出，完成后显示引用溯源芯片，点击后联动左侧 PDF 阅读器跳转到原文位置。

文献列表面板原型

[AI 问答] [文献] [快照] [×]

[搜索作者、标题、关键词] [筛]

共 34 条 • 已匹配 PDF:

He et al. (2024)	
LLM-based Retrieval	
he2024llm • PDF 已匹配	
Chen et al. (2023)	
A Survey of RAG	
chen2023rag • 待匹配 PDF	
[导入 BibTeX] [同步 Zotero]	

文献列表面板采用高密度列表，每行显示作者年份、标题、引用键和 PDF 匹配状态。引用键使用等宽字体，支持复制、双击插入和拖拽到 Markdown 编辑器。

快照时间轴面板原型

[AI 问答] [文献] [快照] [×]	
共 47 个快照 • 占用 12.4 MB	
● 16:35 当前	
○ 16:30 • +42 -8 行	
○ 16:25 • +15 -3 行	

| ● 基线快照 16:10 |

| |

| 预览：16:30 快照 |

| # 第三章

方法论 |

| 本研究采用 |

| [显示差异] [恢复到此版本] |

| |

快照时间轴面板用于提供版本安全感。用户选择时间轴节点后，下方预览区以只读方式展示历史版本；执行恢复前必须弹出确认框，并在恢复前自动创建保护性快照。

3.3.3 关键操作界面设计

关键操作界面设计围绕用户最关心的高风险或高价值操作展开，包括导出文档、AI 问答、公式 OCR、插入引用与恢复快照。

1. 导出文档界面

导出对话框是系统核心价值的最终触点，应在同一模态窗口内完成“选择模板 → 配置输出 → 引文校验 → 执行导出 → 查看结果”的完整闭环。

| |

| 导出文档 [×] |

| |

| 1. 选择排版模板 |

| (●) 武汉大学硕士论文 v2.1 Schema 通过 |

| () IEEE 会议论文 v1.3 Schema 通过 |

| () 自定义模板 v0.2 Schema 失败 |

模板路径: ~/WeaveDoc/templates [管理模板]

2. 导出配置

输出格式: (●) Word (.docx) () PDF

输出文件: [~/Papers/thesis.docx] [浏览]

3. 引文校验

34 条引用全部匹配, 正文与文末参考文献一致

4. 导出进度

解析模板 → 生成 IR → 重排引文 →

Pandoc 转换

[取消] [导出]

导出界面的关键要求如下:

模板列表为单选, Schema 校验失败的模板禁用并显示错误原因。

输出路径必须包含文件名和扩展名, 未选择路径时导出按钮禁用。

引文校验区分成功、普通警告和结构性错误。普通未匹配引用可由用户选择继续导出; 结构性错误必须阻断导出。

导出进行中锁定配置项, 显示阶段式进度: 解析模板、生成 IR、重排引文、Pandoc 转换。

导出失败时明确提示原始 Markdown 文稿未被修改, 降低用户对内容丢失的焦虑。

2. AI 文献问答界面

AI 问答界面位于右侧上下文面板中, 不以模态窗口阻塞写作。用户在输入问题后, 系统在同一面板内展示检索、推理和流式回答结果。

区域	设计内容	关键反馈
模型状态条	模型名、状态灯、推理速度、内存占比	未配置或未加载时引导至设置；加载中显示进度
对话区	用户问题气泡、AI 回答气泡、流式文本	逐 Token 输出，用户滚动历史时显示回到底部按钮
引用溯源区	[1] p.3 形式的引用芯片	点击后 PDF 阅读器跳转并高亮原文
输入区	多行文本框、发送按钮、停止生成按钮	Enter 发送，Shift+Enter 换行，推理中可停止

3. 公式 OCR 界面

公式 OCR 采用 PDF 阅读器上方的上下文浮层，不打断当前阅读和写作流程。

PDF 页面

公式选区

E

= mc^2

▼

公式识别 [×]

正在识别公式

\begin{equation}...

[复制 LaTeX] [重新框选]

浮层紧贴用户框选区域显示。识别中必须展示阶段文案和进度；模型未加载时由本次显式框选操作触发加载；识别成功后展示可复制的 LaTeX 代码；复制成功通过 toast 提示。

4. 快照恢复界面

快照恢复属于破坏性操作，界面必须提供预览、确认和保护性快照机制。

步骤	界面表现	说明
选择快照	时间轴节点高亮，预览区显示只读 Markdown	用户先确认历史内容
点击恢复	恢复按钮使用破坏性样式	提醒用户当前内容将被覆盖
二次确认	ConfirmDialog 显示覆盖风险和保护性快照说明	未确认前不修改编辑器内容
执行恢复	按钮进入 loading，系统先保存当前版本快照	完成后刷新时间轴和编辑器
恢复完成	当前版本节点更新，旧版本仍可回滚	保持版本链可追溯

3.4 界面交互设计

界面交互设计遵循“主流程少跳转、辅助功能可折叠、危险操作可逆、耗时操作可见”的原则。高频工作尽量在工作台内完成，低频配置通过模态集中处理，跨面板联动需要保持明确的视觉反馈。

3.4.1 交互流程说明

1. 写作与导出流程

打开工作区

→ 新建或打开 Markdown 文档

→ 编写正文、公式、表格

- 输入或插入引用键
- 系统匹配 BibTeX 条目并显示预览
- 点击导出
- 选择 AFD 模板、输出格式和路径
- 系统执行引文一致性校验
- 校验通过后执行导出管线
- 显示导出结果或错误详情

该流程的交互重点是：编辑期间不频繁重排正式引用编号，避免正文闪烁；导出前集中完成模板校验和引文一致性校验；导出失败时明确说明原稿安全。

2. 双屏阅读与锚点插入流程

打开 Markdown 文档和关联 PDF

- 用户在 PDF 中定位目标段落或图表
- 点击目标区域
- 系统生成包含页码和区域位置的深层链接锚点
- 在 Markdown 当前光标处插入锚点链接
- 用户点击 Markdown 锚点
- PDF 阅读器跳转到对应页面并高亮原文位置

该流程在主工作台内完成，不打开新窗口。若 PDF 为扫描件且无法生成文本级锚点，则降级为页码级锚点并提示用户。

3. 离线 AI 问答流程

导入 PDF 文献

- 系统提取文本并构建向量索引

- 用户打开 AI 问答面板
- 模型状态条显示未配置、加载中或就绪状态
- 用户输入问题
- 系统检索 Top-N 原文片段
- 本地 LLM 流式生成回答
- 回答底部显示引用溯源芯片
- 用户点击芯片跳转 PDF 原文

AI 问答不能阻断 Markdown 编辑器。模型加载、向量化索引和推理过程均需要明确状态提示，避免用户误认为系统卡死。

4. 公式提取流程

用户在 PDF 中拖拽框选公式区域

- 系统显示蓝色半透明选区
- 松开鼠标后弹出公式识别浮层
- 本地 OCR 模型识别选区截图
- 浮层展示 LaTeX 代码
- 用户点击复制
- 系统提示已复制到剪贴板
- 用户在 Markdown 编辑器中粘贴使用

该流程为上下文浮层交互，不弹出模态窗口。若识别失败，应在浮层内给出重新框选入口。

5. 快照恢复流程

用户打开快照时间轴面板

- 浏览时间轴节点

- 点击某个历史快照
- 预览区显示只读 Markdown 内容
- 用户点击恢复到此版本
- 系统弹出二次确认
- 用户确认后系统先创建保护性快照
- 编辑器内容替换为历史版本
- 时间轴刷新并标记当前版本

快照恢复必须强调可逆性。界面应明确说明“恢复前会自动保存当前版本”，并在恢复完成后保留旧快照节点。

3.4.2 界面跳转关系

WeaveDoc 的界面跳转关系以工作台为中心。除首次配置向导外，所有页面入口都从工作台触发，或在工作台内通过面板、浮层和模态展开。

起始界面	目标界面	触发方式	跳转类型
首次启动	首次配置向导	系统自动检测首次运行	模态向导
首次配置向导	工作台	完成配置或跳过可选项	进入主页面
工作台	AI 问答面板	工具栏 AI、View → AI、Ctrl+Shift+A	同窗口面板切换
工作台	文献列表面板	View → Literature、状态栏 Refs、Ctrl+Shift+L	同窗口面板切换
工作台	快照时间轴面板	View → Snapshot、Ctrl+Shift+T	同窗口面板切换
工作台	导出对话框	工具栏导出、Export → Export Document、Ctrl+Shift+E	模态窗口
工作台	设置页面	File → Settings、Ctrl+,、模型状态条点击	模态窗口
工作台	帮助 / 关于	Help → About、Help → Keyboard Shortcuts、F1	模态窗口
PDF 阅读器	公式提取浮层	鼠标拖拽框选公式区域	上下文浮层
AI 问答面板	PDF 阅读器	点击回答中的引用溯源芯片	跨面板联动
Markdown 编辑器	PDF 阅读器	点击 Markdown 锚点链接	跨面板联动
PDF 阅读器	Markdown 编辑器	点击 PDF 段落或图表	跨面板插入
导出对话框	设置页面模板库	点击“管理模板”	模态切换或打开设置并定位模板库
快照时间轴面板	设置页面快照策略	点击快照策略说明	模态窗口并定位快照策略 Tab

界面跳转应遵循以下规则：

同一任务上下文内优先使用面板切换，不打开新窗口。

配置类操作统一进入设置页面，避免在多个界面分散配置项。

模态窗口关闭后返回打开前的工作台状态，保持 PDF、Markdown 和上下文面板状态不丢失。

跨面板跳转必须提供高亮反馈，例如 PDF 原文区域高亮、Markdown 光标定位或列表行选中。

3.5 界面设计 UML 模型

本节仅保留界面 UML 模型的分析说明。具体 UML 代码已从正文中抽离为独立 PlantUML 源文件，便于后续单独渲染、版本维护和插入最终设计说明书。

PlantUML 源文件如下：

模型	PlantUML 文件	说明
界面导航图	界面导航图.puml	描述启动、主工作台、上下文面板、模态窗口与浮层之间的界面流向
边界类图	边界类图.puml	描述界面边界类、容器组合关系和跨界面依赖
导出文档顺序图	顺序图 - 导出文档.puml	描述从打开导出对话框到完成导出或处理校验异常的交互流程
AI 文献问答顺序图	顺序图 - AI 文献问答.puml	描述 AI 面板打开、模型状态检查、RAG 检索、流式回答和 PDF 溯源跳转
公式 OCR 顺序图	顺序图 - 公式 OCR.puml	描述 PDF 框选公式、浮层识别、复制 LaTeX 和异常处理
快照恢复顺序图	顺序图 - 快照恢复.puml	描述快照选择、预览、二次确认、保护性快照和版本恢复

3.5.1 界面导航图

界面导航图用于说明 WeaveDoc 的整体界面组织关系。系统采用以工作台为中心的单窗口结构，首次配置向导只在首次启动时进入，完成后回到主工作台。用户日常操作均从工作台出发，通过工具栏、菜单栏、快捷键、状态栏或局部交互进入对应功能界面。

从导航层级看，工作台是唯一常驻主页面；AI 问答、文献列表和快照时间轴属于右侧上下文面板中的同级 Tab；导出、设置、首次配置向导和帮助 / 关于属于模态界面；公式 OCR 属于 PDF 阅读器上的上下文浮层。该组织方式减少了页面跳转数量，使写作、阅读和 AI 辅助能够在同一工作上下文中连续完成。

导航图还体现了两个重要的跨面板联动关系：其一，AI 回答中的引用溯源可以跳转并高亮 PDF 原文；其二，Markdown 中的锚点链接可以反向定位 PDF 页面，而 PDF 侧点击段落又可以向 Markdown 当前光标处插入锚点。通过这些联动，系统将文献阅读与论文写作连接为一个闭环。

3.5.2 边界类图

边界类图用于说明界面层的主要边界对象及其组合关系。WorkspaceWindow 是主边界类，负责承载菜单栏、工具栏、PDF 阅读器、Markdown 编辑器、右侧上下文面板和底部状态栏，是整个界面结构的聚合根。

右侧上下文面板由 ContextPanelBoundary 统一管理，内部组合 ChatPanelBoundary、LiteraturePanelBoundary 和 SnapshotPanelBoundary 三个子边界类。这样的划分可以保证三个辅助功能共享统一的折叠、展开和 Tab 切换规则，同时又能在各自边界内维护独立的状态、输入和反馈。

模态类界面由独立边界类表示，包括 ExportDialogBoundary、SettingsDialogBoundary、SetupWizardBoundary 和 ConfirmDialogBoundary。其中导出对话框与设置页面存在“管理模板”跳转关系，快照面板与确认对话框存在恢复确认关系。公式 OCR 使用 OCRFloatingToolbarBoundary 表示，它不是全局模态，而是由 PDF 框选区域触发的局部浮层。

该边界类划分只描述界面层职责，不直接包含导出服务、RAG 检索、OCR 模型、快照存储等业务对象。业务对象在顺序图中以控制类或数据存储参与者出现，避免界面类图承担过多业务细节。

3.5.3 关键界面交互顺序图

关键界面交互顺序图覆盖四条最能体现系统价值和风险控制的操作路径：导出文档、AI 文献问答、公式 OCR 和快照恢复。

导出文档顺序图强调导出前校验与导出中反馈。用户打开导出对话框后，先完成模板、格式和路径选择，再触发引文一致性校验。校验通过时进入导出管线；普通未匹配引用以警告方式提示，允许用户选择返回修改或继续导出；结构性错误直接禁用导出按钮。该流程保证导出结果的确定性，并在失败时明确提示原稿安全。

AI 文献问答顺序图强调模型状态可见和回答可溯源。用户打开 AI 面板后，界面先检查模型状态；如果模型未加载且允许自动加载，则展示加载进度；如果模型未配置，则引导用户进入设置页面。问题发送后，系统完成向量检索、Prompt 组装和本地 LLM 推理，回答以流式方式展示，并在末尾给出引用溯源芯片。用户点击芯片后，PDF 阅读器跳转到对应页码并高亮原文。

公式 OCR 顺序图强调轻量浮层与即时复制。用户在 PDF 中框选公式区域后，系统在选区附近弹出公式提取浮层，并调用本地 OCR 模型识别 LaTeX 代码。识别成功后，用户可一键复制并粘贴到 Markdown 编辑器；模型未加载时显示加载进度并自动重试；识别失败时保留在浮层内提示原因和重新框选入口。

快照恢复顺序图强调破坏性操作的可逆控制。用户在快照时间轴中选择历史节点后，系统先重建并展示只读预览；点击恢复时必须经过确认对话框。用户确认后，系统先保存当前编辑器内容为保护性快照，再用历史版本替换当前内容，并刷新时间轴。该流程保证恢复操作不会让当前版本不可追溯地丢失。

4 详细设计

4.1 用例详细设计

基于 MoSCoW 优先级和系统复杂度分析，选取以下 3 个核心用例进行详细设计：

用例编号	用例名称	涉及需求	优先级	复杂度
UC-01	学术文档高保真导出	FR-01 + FR-03	Must Have	最高
UC-02	离线 RAG 文献问答	FR-02	Must Have	高
UC-03	双屏联动编辑与预览	FR-04	Should Have	中高

4.1.1 用例 1：学术文档高保真导出

基本信息

项目	说明
用例名称	学术文档高保真导出
参与者	研究者 / 学生
前置条件	(1) 用户已编写完成 Markdown 文档； (2) 系统已通过 ConfigManager 加载至少一个 AFD 样式模板； (3) 本地已安装 Pandoc 引擎
后置条件	在指定路径生成符合高校论文格式规范的 .docx 或 .pdf 文件
触发事件	用户在 ConvertTab 界面选择模板、格式和输出路径后点击”转换”

详细执行步骤

步骤	执行者	动作描述
1	用户	选择 MD 输入文件、AFD 模板、导出格式（DOCX/PDF）
2	ConvertTab	校验输入参数（文件存在性、输出路径合法性）
3	DocumentConversionEngine	调用 ConfigManager.GetTemplateAsync(templateId) 加载模板
4	TemplateRepository	从 SQLite templates 表查询模板元数据
5	AfdParser	读取 JSON 文件，反序列化为 AfdTemplate，校验必填字段
6	ReferenceDocBuilder	使用 OpenXML API 构建参考文档 reference.docx（页面设置 + 样式定义 + 眉页脚）
7	MarkdownHtmlTableNormalizer	HTML 表格转为 pipe 表格
8	MarkdownMathNormalizer	LaTeX \textcircled{} 转为 Unicode 圈号
9	MarkdownHtmlImageNormalizer	 标签转为 Markdown 图片语法
10	PandocPipeline	发现 Lua 筛选器（assign-block-styles.lua、afd-heading-filter.lua）
11	PandocPipeline	组装 CLI 参数，启动 Pandoc 子进程：pandoc --from markdown --to docx --reference-doc=ref.docx --lua-filter=xxx.lua
12	PandocPipeline	等待子进程完成，捕获 ExitCode 和输出
13	OpenXmlStyleCorrector	ApplyAfdStyles() - 遍历 DOCX 段落，匹配 AFD 样式定义，移除冗余性
14	OpenXmlStyleCorrector	ApplyHeaderFooter() - 写入页眉页脚内容
15	OpenXmlStyleCorrector	ApplyPdfLayout() - 处理双栏布局、前置页检测、宽表格跨栏（仅 PDF 模式）
16	CompositePdfConverter	按策略链尝试 PDF 转换：Word COM → LibreOffice → Syncfusion（失败则回退到 Word COM 模式）
17	DocumentConversionEngine	清理临时文件（预处理 MD、reference.docx）
18	ConversionErrorMessage	若发生异常，格式化为用户友好的中文提示
19	ConvertTab	显示导出结果（成功路径 / 错误信息）

异常处理

异常场景	检测条件	处理方式
模板不存在	TemplateRepository 返回 null	提示” 模板不存在，请先导入模板”
模板 JSON 格式错误	AfdParser 抛出 AfdParseException	提示” 模板格式错误”
Pandoc 未安装	PandocPipeline 无 法解析 pandoc 路径	提示” 未找到 Pandoc，请安装”
Pandoc 转换失败	ExitCode ≠ 0	PandocException → ConversionErrorFormatter 生成中 示
PDF 转换全部失败	CompositePdfConv erter 所有策略均失败	提示” PDF 转换失败，请安装 Word 或 LibreOffice
输入 MD 文件不存在	ConvertTab 校验失 败	提示” 文件不存在”

关键算法：AFD 样式映射算法

```

1  算法: ApplyAfdStyles(docxPath, template)
2  输入: docxPath: DOCX文件路径, template: AFD模板
3  输出: 样式修正后的DOCX文件
4
5  1.  doc ← 用 OpenXML 打开 docxPath
6  2.  styleMap ← AfdStyleMapper 预定义的 AFD键 ↔ OpenXML styleId 双向映射表
7  3.
8  4.  FOR EACH paragraph IN doc.Body.Descendants<Paragraph>():
9  5.      currentStyle ← paragraph.ParagraphProperties?.ParagraphStyleId
10 6.
11 7.      IF currentStyle IN styleMap:
12 8.          afdKey ← styleMap[currentStyle]  // 反查 AFD 键名
13 9.          IF afdKey IN template.Styles:
14 10.              def ← template.Styles[afdKey]
15 11.              应用字体: def.FontFamily → RunProperties.Fonts
16 12.              应用字号: def.FontSize → RunProperties.FontSize
17 13.              应用粗体: def.Bold → RunProperties.Bold
18 14.              应用段前距: def.SpaceBefore → ParagraphProperties.SpacingBefore
19 15.              应用首行缩进: def.FirstLineIndent → ParagraphProperties.Indentation
20 16.          END IF
21 17.      END IF
22 18.
23 19.      // 移除 Pandoc 遗留的冗余内联属性
24 20.      StripInlineProperties(paragraph)
25 21. END FOR
26
27 23. doc.Save()

```

4.1.2 用例 2: 离线 RAG 文献问答

基本信息

项目	说明
用例名称	离线 RAG 文献问答
参与者	研究者 / 学生
前置条件	(1) 本地 GGUF 格式 Embedding 模型和 Chat 模型已下载； (2) RagOptions 环境变量已配置
后置条件	聊天面板展示 AI 回答，回答基于本地文献内容
触发事件	用户在 RagTab 面板输入问题并提交

详细执行步骤

步骤	执行者	动作描述
1	用户	切换到 RAG 问答标签页
2	RagTabViewModel	调用 LocalAiService.InitializeAsync()
3	LocalAiService	RagOptions.LoadFromEnvironment() 加载配置
4	LocalAiService	LLamaWeights.LoadFromFileAsync() 加载 Embedding
5	LocalAiService	LLamaEmbedder 实例化，用于后续向量化
6	LocalAiService	扫描文档目录，对每个文档执行分块（MarkdownChunking JsonChunking）
7	LocalAiService	对每个文本块计算 Embedding 向量，缓存到 EmbeddingCache
8	LlamaServerProcess	启动 Reranker llama-server 进程（--reranking 模式）
9	RagTab	提示” AI 助手已就绪”
10	用户	输入问题
11	QueryUnderstandingService	BuildProfile() - 正则匹配意图（explain/compare/procedure/metadata...）、提取焦点术语、
12	LocalAiService	LLamaEmbedder 计算问题向量
13	LocalAiService	在 EmbeddingCache 中计算余弦相似度，取 Top-K 候
14	CandidateRetriever	HTTP POST 到 Reranker 服务获取重排序分数
15	ChunkRanker	根据查询意图计算加权分数（procedure/compare/definition 等）
16	AnswerComposer	组装 System Prompt（角色定义 + 参考内容 + 问题 + 对话历史）
17	LlamaServerChatClient	HTTP POST /v1/chat/completions 到本地 llama-server
18	LlamaServerChatClient	接收 SSE 流式响应，逐 token 返回
19	RagTabViewModel	实时更新 ConversationText
20	LocalAiService	AnswerPostProcessing - 引用归一化、离题检测、修复拼写错误
21	RagTab	显示完整回答

异常处理

异常场景	检测条件	处理方式
Embedding 模型文件不存在	LoadFromFileAsync 抛出异常	提示” 模型文件未找到，请检查路径”
Reranker 服务不可用	HTTP 请求超时 / 失败	跳过重排序，使用原始相似度分数
llama-server 未启动	EnsureServerAvailable Async 检测失败	尝试启动进程；若失败则提示” 推理服务不可用”
响应被截断	finish_reason: "length"	自动发送续写请求（continuation），直到完成或达到最大长度
云端 API 不可用	HTTP 错误响应	降级到本地推理或提示” 云端服务不可用”



关键算法：余弦相似度检索算法

```
1 算法: FindRelevantChunks(queryEmbedding, cache, topK)
2 输入: queryEmbedding: 问题向量(float[]), cache: EmbeddingCache, topK: 返回
   数量
3 输出: topKChunks: 最相关的 topK 个 DocumentChunk
4
5 1. results ← 空优先队列(按相似度降序)
6 2. queryNorm ← SQRT(SUM(queryEmbedding[i]2))
7 3.
8 4. FOR EACH entry IN cache.Entries:
9 5.     dotProduct ← 0
10 6.     vecNorm ← 0
11 7.     FOR i ← 0 TO dimension-1:
12 8.         dotProduct += queryEmbedding[i] entry.Embedding[i]
13 9.         vecNorm += entry.Embedding[i]2
14 10.    END FOR
15 11.    vecNorm ← SQRT(vecNorm)
16 12.    similarity ← dotProduct / (queryNorm vecNorm)
17 13.
18 14.    IF results.size < topK OR similarity > results.peekMin():
19 15.        results.push((entry, similarity))
20 16.        IF results.size > topK:
21 17.            results.popMin()
22 18.        END IF
23 19.    END IF
24 20. END FOR
25 21.
26 22. RETURN results 按相似度降序排列
```

4.1.3 用例 3：双屏联动编辑与预览

基本信息

项目	说明
用例名称	双屏联动编辑与预览
参与者	研究者 / 学生
前置条件	(1) 用户已切换到编辑器标签页； (2) WebView2 运行时已初始化
后置条件	编辑器 + 预览 + PDF 查看器三屏协同工作
触发事件	用户切换到编辑器标签页

详细执行步骤

步骤	执行者	动作描述
1	用户	切换到”编辑器”标签页
2	MarkdownEditorTab	ActivateAsync() 激活三个 WebView2 控
3	MonacoEditorControl	加载 monaco-editor/index.html，初始化结
4	PreviewWebViewControl	加载 preview-template.html，注册 WeaveD COM 对象
5	PdfViewerControl	启动本地 HTTP 服务，加载 PDF.js viewer.
6	用户	打开 .md 文件
7	MonacoEditorControl	通过 PostWebMessageAsJson 设置编辑器
8	MarkdownService	ConvertMarkdownToHtmlWithCharPositions 换，插入 data-line/data-pos 属性
9	PreviewWebViewControl	调用 JS updatePreview(html) 更新预览
10	用户	编辑 Markdown 文本
11	MonacoEditorControl	捕获 onDidChangeModelContent 事件
12	MarkdownService	实时 MD→HTML 转换
13	PreviewWebViewControl	增量更新预览
14	用户	滚动编辑器
15	MonacoEditorControl	捕获滚动事件，传递行号
16	PreviewWebViewControl	JS 查找 [data-line="N"] 元素，scrollIntoV
17	用户	选中文字
18	MonacoEditorControl	捕获 onDidChangeCursorSelection 事
19	PreviewWebViewControl	JS 高亮对应预览区域
20	用户	打开 .pdf 文件
21	PdfViewerControl	通过 PDF.js 加载并渲染 PDF

异常处理

异常场景	检测条件	处理方式
WebView2 未安装	运行时初始化失败	提示用户安装 WebView2
Monaco 加载超时	WebView2 导航超时	重试加载，显示”编辑器加载中“
MD→HTML 转换失败	MarkdownService 正则匹配异常	显示原始文本，记录错误
PDF.js 加载失败	PDF 文件损坏或格式不支持	提示”PDF 文件无法打开“
标签页切换	用户切换离开编辑器	DeactivateAsync() : 关闭 WebView2，节省资源

关键算法：Markdown→HTML 带行号标记转换

```

1 算法: ConvertMarkdownToHtmlWithCharPositions(markdown)
2 输入: markdown: Markdown文本
3 输出: html: 带data-line和data-pos属性的HTML
4
5 1.  lines ← markdown.Split('\n')
6 2.  html ← ""
7 3.
8 4.  FOR i ← 0 TO lines.Length - 1:
9 5.      line ← lines[i]
10 6.      lineNum ← i + 1
11 7.
12 8.      IF line 匹配 "^#{1,6}\s":
13 9.          level ← 标题级别(1-6)
14 10.          text ← 去除 # 前缀
15 11.          html += "<h{level} data-line=\"{lineNum}\" data-pos=\"0\">
                {text}</h{level}>"
16 12.
17 13.      ELSE IF line 匹配 "^```":
18 14.          // 代码块处理
19 15.          收集直到下一个 ``` 的所有行
20 16.          html += "<pre data-line=\"{lineNum}\"><code>...</code></pre>"
21 17.
22 18.      ELSE IF line 匹配 "^\\|.*\\|":
23 19.          // 表格行处理
24 20.          html += "<td data-line=\"{lineNum}\">...</td>"
25 21.
26 22.      ELSE:
27 23.          // 普通段落
28 24.          html += "<p data-line=\"{lineNum}\" data-pos=\"0\">{line}</p>"
29 25.      END IF
30 26. END FOR
31 27.
32 28. RETURN html

```

4.2 类设计

4.2.1 类识别与定义

系统共有 **40+ 个核心类**，按子系统分组如下：

子系统	核心类	
WeaveDoc.App	Program, App, MainWindow, ConvertTab, TemplateTab, RagTabViewModel	
WeaveDoc.MarkdownEditor	MarkdownEditorTab, MonacoEditorControl, PreviewWebViewControl, PdfViewerControl, MainWindowViewModel, MonacoEditorViewModel, MarkdownService	
WeaveDoc.Converter	DocumentConversionEngine, PandocPipeline, IPdfConverter, CompositePdfConverter, SyncfusionPdfConverter, WordComPdfConverter, LibreOfficePdfConverter, PdfRendererDetector, ReferenceDocBuilder, OpenXmlStyleCorrector, AfdParser, AfdStyleMapper, ConfigManager, TemplateRepository, BibtexParser, ConversionErrorFormatter, 3 个 Normalizer	
WeaveDoc.Converter.Afd.Models	AfdTemplate, AfdMeta, AfdDefaults, AfdStyleDefinition, AfdHeaderFooter, BibtexEntry 等	
WeaveDoc.Rag	LocalAiService, LlamaServerChatClient, LlamaServerProcess, AnswerComposer, CandidateRetriever, ChunkRanker, EvidenceSelector, QueryUnderstandingService, LocalAiQuestionAnalyzer, CloudApiSettings, RagOptions	
WeaveDoc.Rag.Models	ChatTurn, DocumentChunk, QueryProfile, ScoredChunk 等	

4.2.2 类关系说明

组合关系（Composition *--）

整体	部分	含义
MainWindow	RagTabViewModel	主窗口持有 RAG 视图模型
MarkdownEditorTab	MonacoEditorControl, PreviewWebViewControl, PdfViewerControl	编辑器标签页包含三个 WebView
AfdTemplate	AfdMeta, AfdDefaults, AfdStyleDefinition, AfdHeaderFooter	AFD 模板由元数据、默认值、样式定义、 眉页脚组成

策略模式（Strategy IPdfConverter）

实现类	平台	说明
WordComPdfConverter	Windows only	COM 自动化驱动 Word
LibreOfficePdfConverter	跨平台	headless soffice 转换
SyncfusionPdfConverter	跨平台	纯 .NET 渲染（兜底）
CompositePdfConverter	-	策略链编排，按优先级尝试

依赖关系（Dependency -->）

源类	目标类 / 接口	含义
DocumentConversionEngine	PandocPipeline, IPdfConverter, ConfigManager	编排完整转换流程
ConfigManager	TemplateRepository, AfdParser	模板 CRUD + 解析
LocalAiService	LLamaEmbedder, LlamaServerChatClient, AnswerComposer, CandidateRetriever, ChunkRanker	编排 RAG 问答逻辑
MainWindowViewModel	MarkdownService	MD→HTML 渲染
PandocPipeline	3 个 MarkdownNormalizer	预处理输入/输出

4.2.3 关键类详细描述

DocumentConversionEngine（文档转换引擎）

项目	说明
职责	编排完整的文档转换管线：加载模板 → 构建参考文档 → 预处理 MD → Pandoc → OpenXML 修正 → PDF 转换
所属项目	WeaveDoc.Converter
构造器依赖	PandocPipeline, IPdfConverter, ConfigManager

核心方法：

Plain Text

```
1 public async Task<ConversionResult> ConvertAsync(  
2     string markdownPath, string templateId, ExportFormat format,  
3     PdfLayoutMode pdfLayoutMode, Cancellation_token ct)  
4 管线流程：  
5     1. ConfigManager.GetTemplateAsync() → AfdTemplate - 加载模板  
6     2. ReferenceDocBuilder.Build() → reference.docx - 构建参考文档  
7     3. MarkdownNormalizer.Normalize() × 3 - 预处理输入  
8     4. PandocPipeline.ToDocxAsync() → DOCX - Pandoc 转换  
9     5. OpenXmlStyleCorrector.ApplyAfdStyles() - 样式修正  
10    6. CompositePdfConverter.ConvertToPdf() - PDF 转换（可选）
```

LocalAiService（本地 AI 服务）

项目	说明
职责	管理本地 Embedding 模型、文档分块与向量化缓存、RAG 检索增强生成全流
所属项目	WeaveDoc.Rag
实现方式	partial class，按逻辑拆分为 8 个文件

核心方法：

Plain Text

```
1 public async Task<string> AskAsync(  
2     string question, List<ChatTurn> history, CancellationToken ct)  
3 RAG 流程:  
4     1. QueryUnderstandingService.BuildProfile() → QueryProfile - 查询理  
   解  
5     2. LLamaEmbedder 计算问题向量  
6     3. 内存 EmbeddingCache 余弦相似度检索 Top-K  
7     4. CandidateRetriever.TryRerankWithLearnedModelAsync() - Reranker 重  
   排序  
8     5. ChunkRanker.ComputeTopChunkIntentBoost() - 意图加权  
9     6. AnswerComposer.BuildPrompt() - 组装 Prompt  
10    7. LlamaServerChatClient.CompleteAsync() → SSE 流式推理
```

PandocPipeline（Pandoc 管线）

项目	说明
职责	管理 Pandoc CLI 子进程的生命周期，包括路径发现、参数组装、进程执行
所属项目	WeaveDoc.Converter

核心方法：

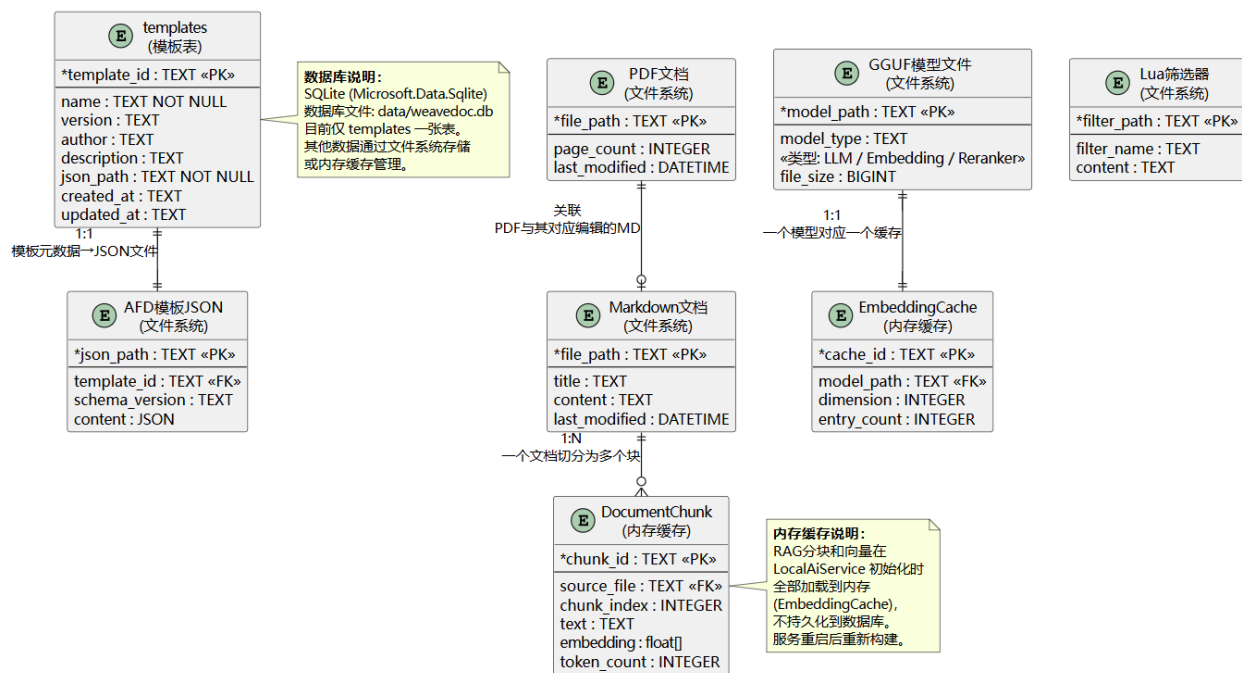
Plain Text

```
1 public async Task ToDocxAsync(string inputPath, string outputPath, stri  
   ng referenceDoc,string luaFilter, CancellationToken ct)  
2 关键设计:  
3     - 自动发现 Pandoc 二进制 (tools/pandoc/ → 上级目录遍历 → 系统 PATH)  
4     - 三个 Normalizer 预处理 (HTML表格、数学公式、图片)  
5     - 自动发现 LuaFilters/ 目录下的 .lua 文件  
6     - PandocException 封装 ExitCode、Stdout、Stderr
```

4.3 数据设计

4.3.1 数据库概念结构设计（E-R 图）

WeaveDoc 数据库 E-R 图



WeaveDoc 的数据存储分为三个层次:

存储层次	技术	数据内容
SQLite 数据库	Microsoft.Data.Sqlite	仅 templates 表 (模板元数据)
文件系统	直接 I/O	MD/PDF 文档、AFD JSON 模板、GGUF 筛选器
内存缓存	EmbeddingCache	RAG 向量索引 (服务重启后重建)

4.3.2 数据库逻辑结构设计

templates 表 (SQLite - 唯一数据库表)

字段名	数据类型	约束	说明
template_id	TEXT	PK	模板唯一标识（如 “default-the:”
name	TEXT	NOT NULL	模板显示名称
version	TEXT		模板版本号
author	TEXT		模板作者
description	TEXT		模板描述
json_path	TEXT	NOT NULL	AFD JSON 文件的存储路径
created_at	TEXT		创建时间
updated_at	TEXT		最后更新时间

Plain Text

```
1 CREATE TABLE IF NOT EXISTS templates (  
2     template_id TEXT PRIMARY KEY,  
3     name TEXT NOT NULL,  
4     version TEXT,  
5     author TEXT,  
6     description TEXT,  
7     json_path TEXT NOT NULL,  
8     created_at TEXT,  
9     updated_at TEXT  
10 );
```

文件系统存储结构

系统运行时的数据存储分布在以下位置：

存储位置	数据内容	管理者
data/weavedoc.db	SQLite 数据库（templates 表）	ConfigManager.TemplateRepository
src/WeaveDoc.Converter/Config/TemplateSchemas/	内置 AFD 模板 JSON（嵌入式资源）	ConfigManager.EnsureTemplatesAsync
doc/markdown/	用户 Markdown 文档	MonacoEditorController
doc/PDF/	用户 PDF 文献文件	PdfViewerController
doc/Word/	导出的 Word 文档	DocumentConversionService
doc/json/	JSON 结构化数据	LocalAiService
.rag/cloud-settings.json	云端 API 配置	CloudApiSettingsLoader
模型目录（由 RagOptions 指定）	GGUF 模型文件 （LLM/Embedding/Reranker）	LocalAiService.LLMManager
系统临时目录	预处理 MD、reference.docx 等中间文件	DocumentConversionService （用后清理）
内存（EmbeddingCache）	RAG 向量索引（服务重启后自动重建）	LocalAiService

内存缓存结构（RAG）

数据结构	类型	内容
List<IndexedChunk>	内存列表	所有文档分块及其 Embedding 向量
EmbeddingCache	内存字典	模型路径→向量缓存映射

IndexedChunk 结构：

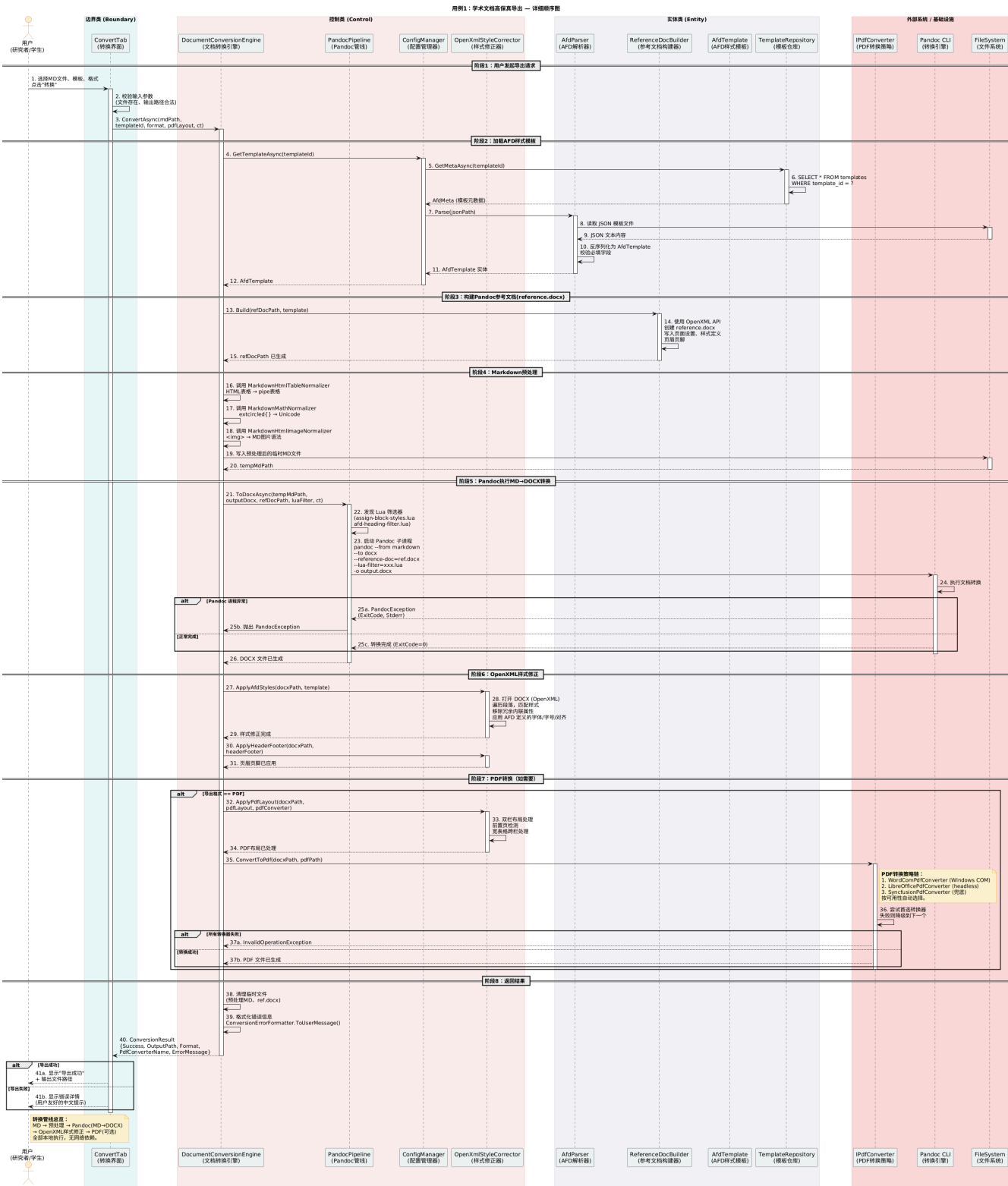
字段	类型	说明
Source	string	来源文件路径
FilePath	string	文件完整路径
Index	int	分块序号
Text	string	分块文本内容
Embedding	float[]	Embedding 向量

4.3.3 数据存储与访问策略

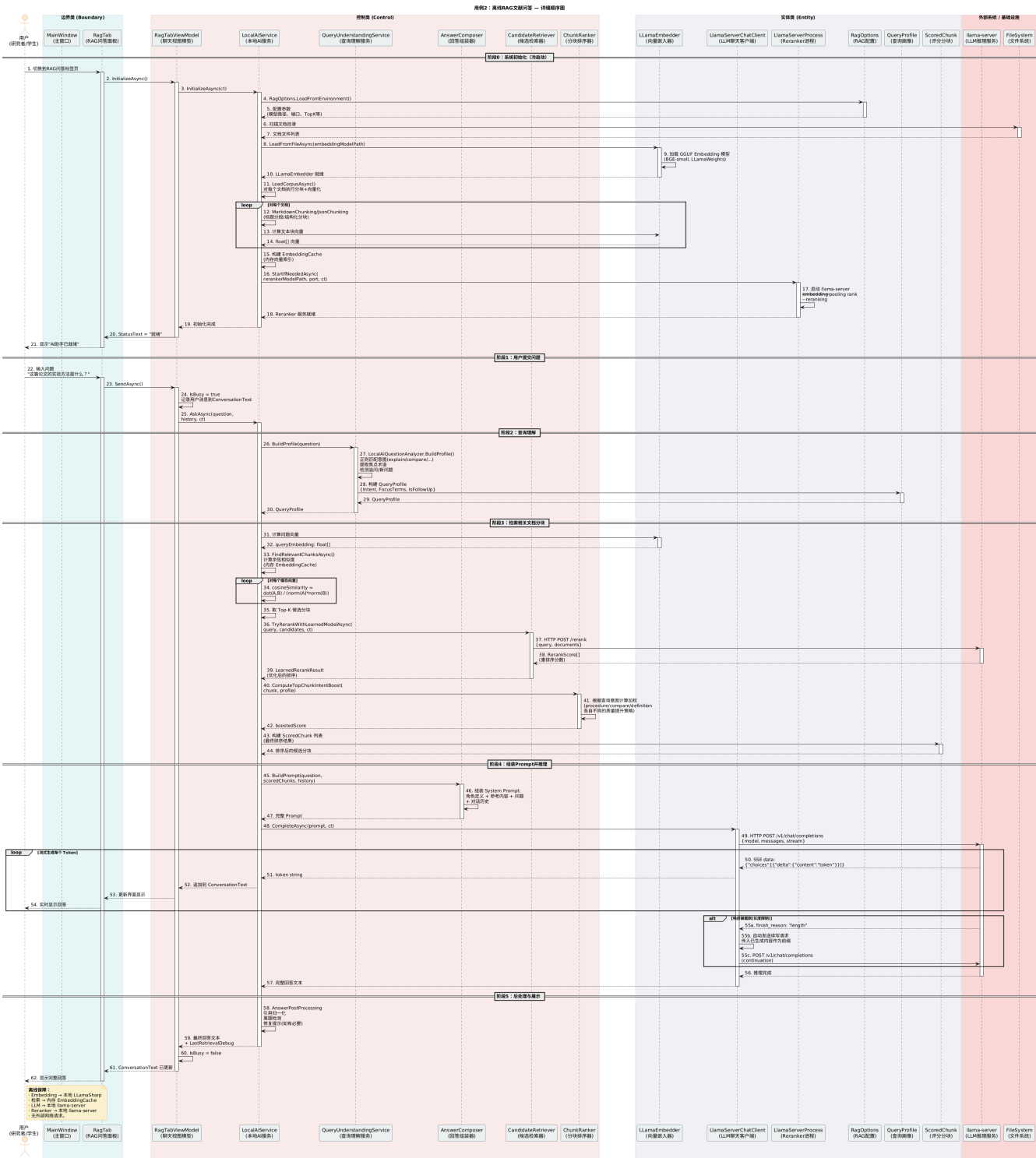
数据类型	存储方式	访问策略	说明
AFD 模板元数据	SQLite (templates 表)	读多写少，CRUD 操作	通过 ConfigManager.TemplateRepository
AFD 模板 JSON	文件系统 (.json)	按需读写	嵌入式资源 + 用户自定义
Markdown 文档	文件系统 (.md)	按需读写	WebView2 Monaco
PDF 文档	文件系统 (.pdf)	只读	PDF.js 渲染
RAG 向量索引	内存 (EmbeddingCache)	服务启动时全量加载	服务重启后自动重新加载
对话历史	内存 (ConversationText)	会话级别	不持久化
GGUF 模型	文件系统 (.gguf)	启动时加载	LLamaWeights + LLaMA
云端 API 配置	文件系统 (JSON)	读写	CloudApiSettings.Local

4.4 详细设计 UML 模型

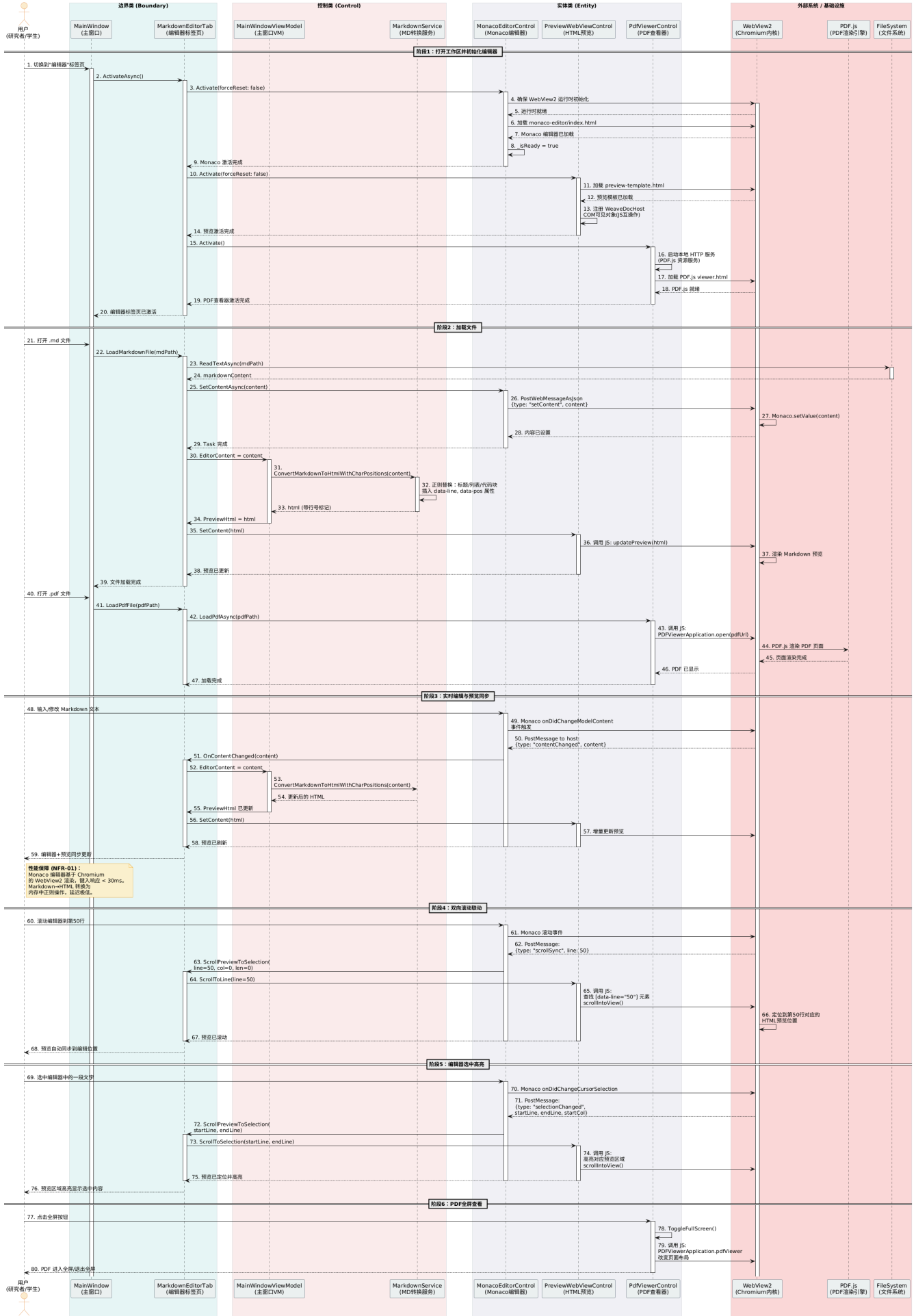
4.4.1 详细类图



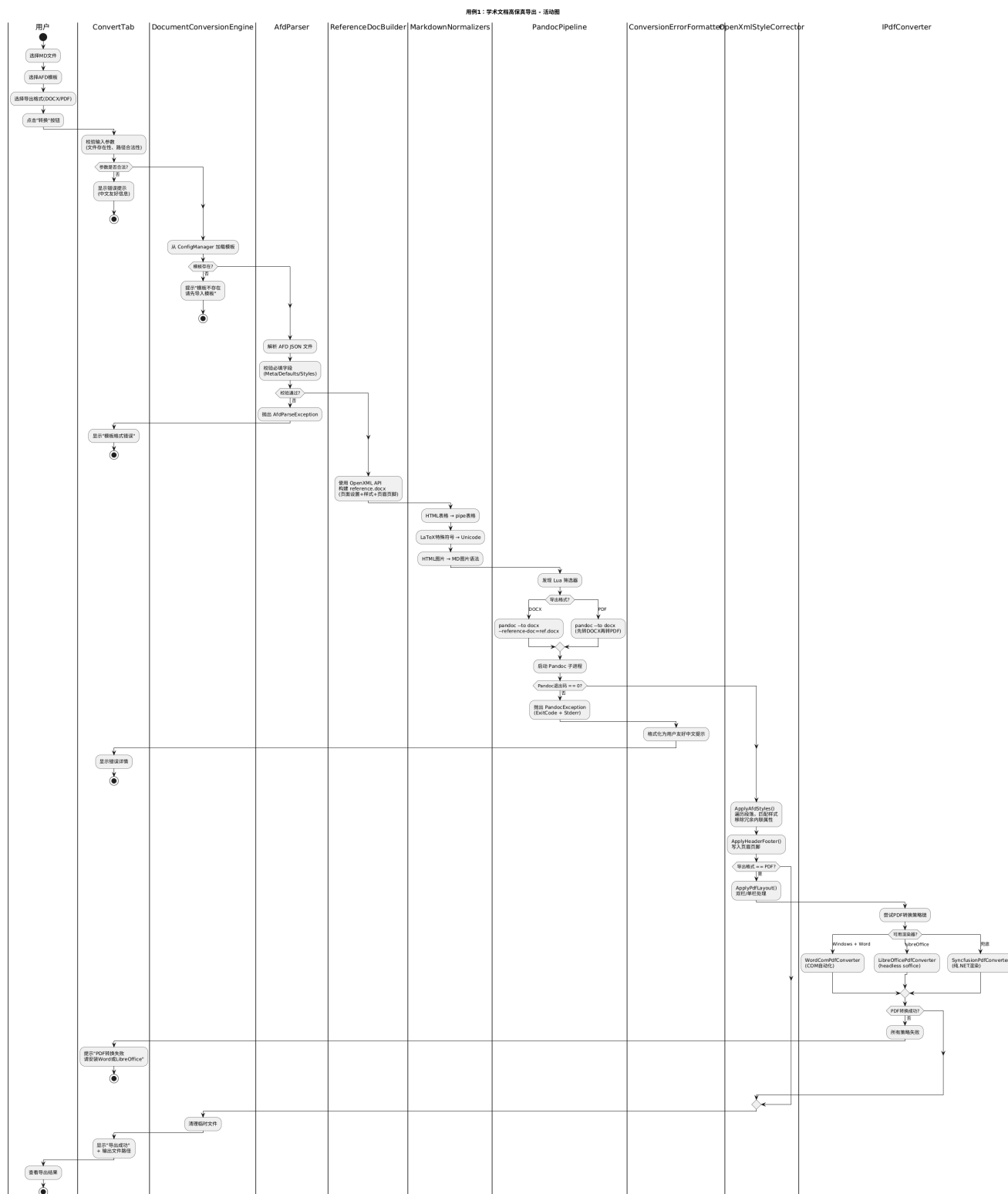
用例2：离线 RAG 文献问答



用例3：双屏联动编辑与预览

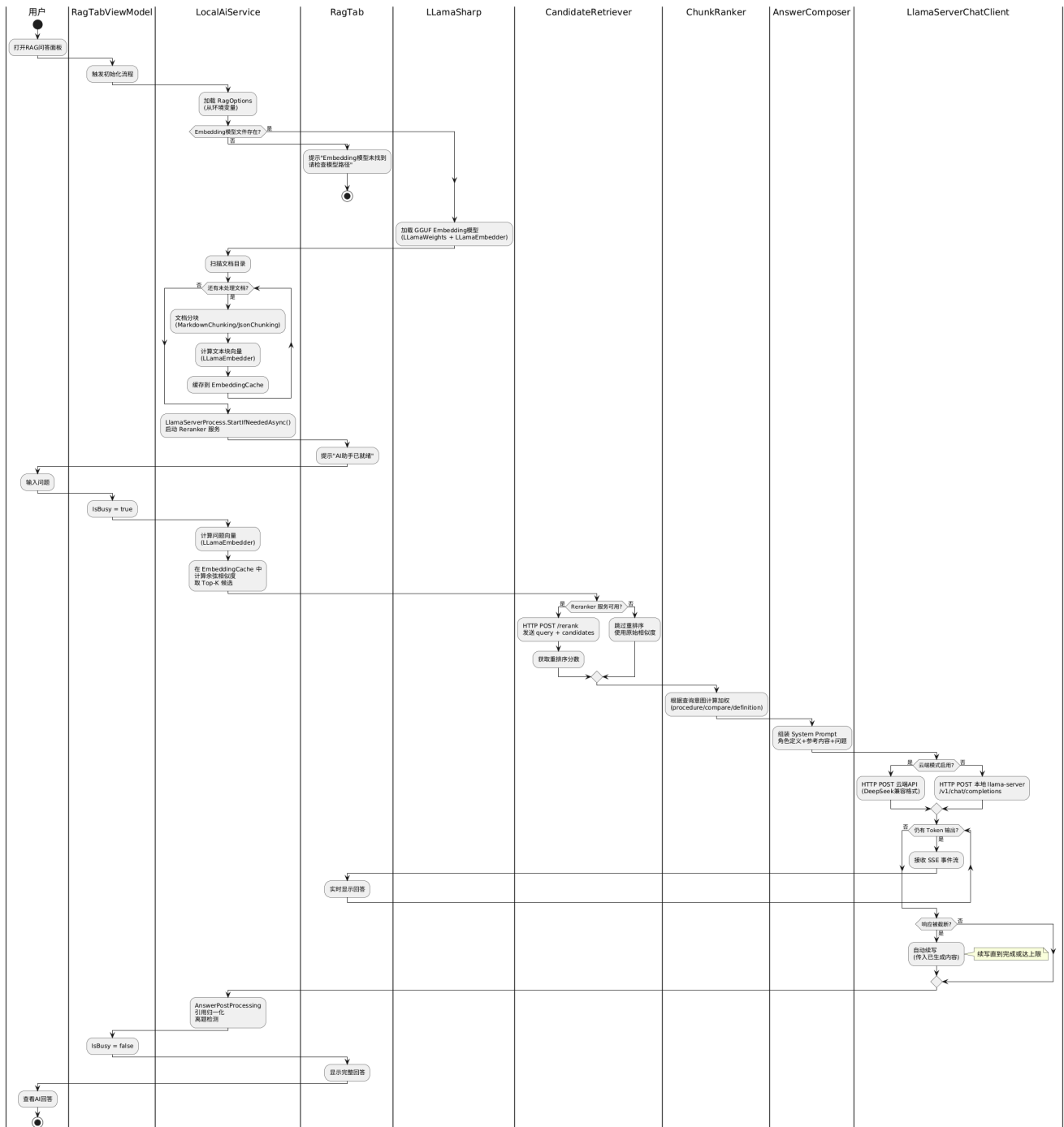


用例 1：学术文档高保真导出

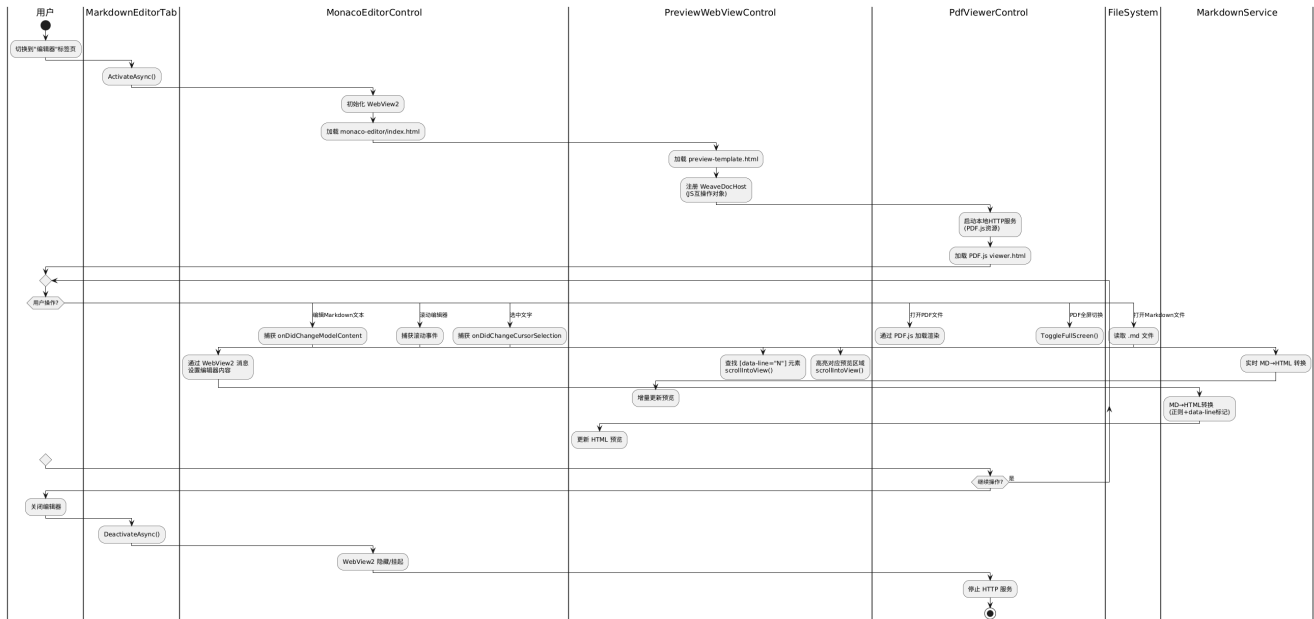


用例 2：离线 RAG 文献问答

用例2：离线RAG文档问答 - 活动图

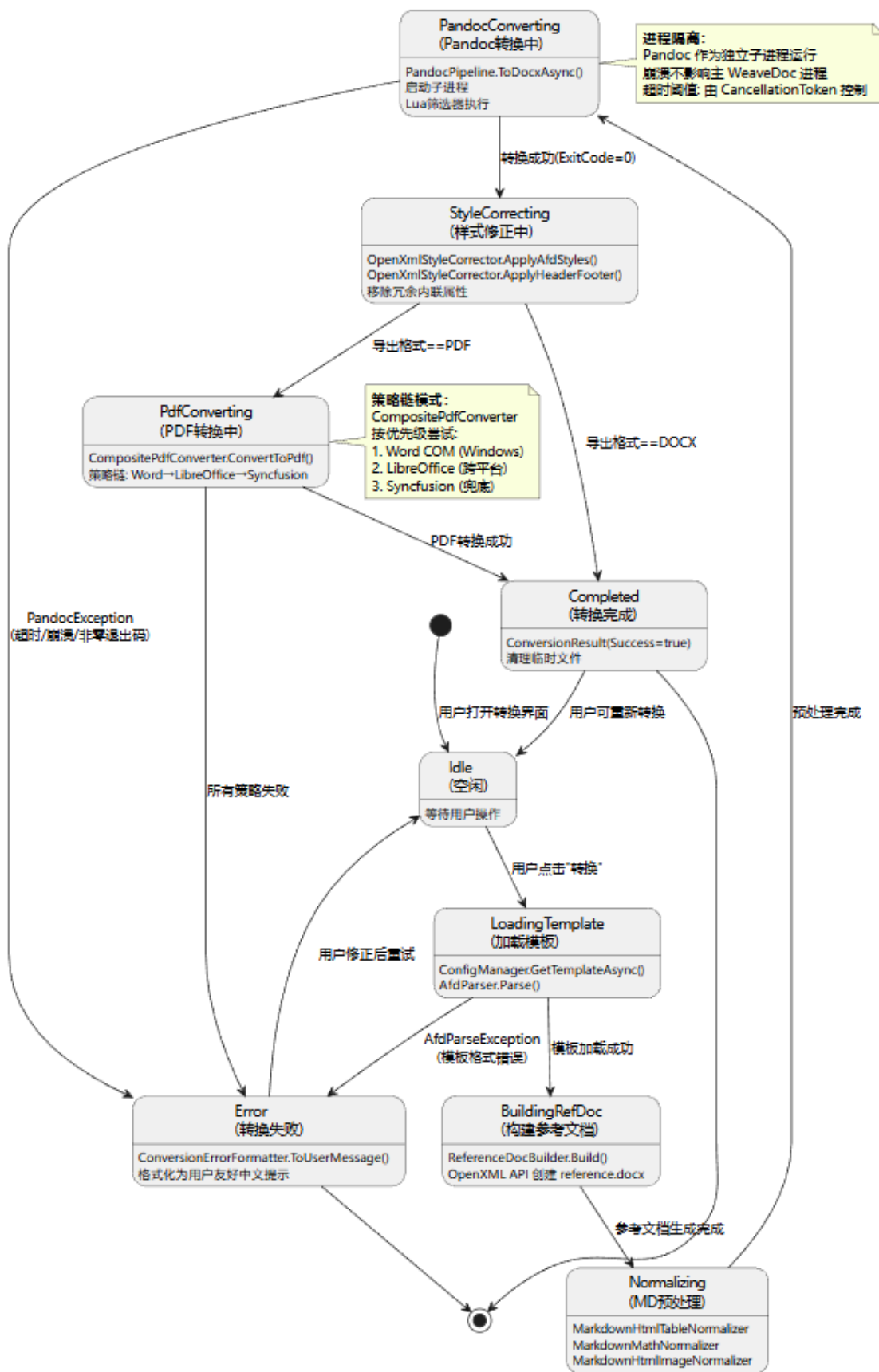


用例3：双屏联动编辑与预览



4.4.4 状态图

文档转换管线状态图 (DocumentConversionEngine)



5 设计总结

5.1 设计成果概述

本设计说明书围绕 WeaveDoc 的“高保真导出、离线 RAG 问答、双屏联动编辑”三大核心用例，完成了完整的架构与详细设计，主要成果包括：

1. **系统架构**：采用 **分层架构 + 策略模式** 的混合设计，将系统划分为 WeaveDoc.App（用户界面层）、WeaveDoc.Converter（文档转换层）、WeaveDoc.Rag（智能问答层）与 WeaveDoc.MarkdownEditor（编辑器层）四个高内聚子系统。子系统间通过接口松耦合调用，支持跨平台策略（如 PDF 转换三策略链）和灵活扩展。
2. **界面设计**：基于高校研究者使用习惯，定位为**专业、简洁、聚焦写作**的桌面应用风格，制定了统一的色彩、字体与布局规范，并完成了主界面、转换标签页、RAG 问答标签页及编辑器三屏布局的原型设计与交互流程说明。
3. **核心用例详细设计**：对三个 Must Have/Should Have 用例的执行步骤、异常处理、关键算法进行了逐项细化。文档转换管线囊括了 AFD 样式映射算法、Pandoc 管线编排与复合 PDF 转换策略；RAG 问答设计了包含查询理解、向量检索、Reranker 重排序、意图加权与流式推理的完整管线；双屏编辑实现了带行号映射的实时 Markdown→HTML 转换与滚动同步算法。
4. **类与数据设计**：识别出 40+ 个核心类，明确了组合、策略与依赖关系，并对 DocumentConversionEngine、LocalAiService、PandocPipeline 三个核心控制类进行了详细职责与方法描述。数据存储采用 SQLite（模板元数据）、文件系统（文档 / 模型）与内存缓存（向量索引）三级策略，兼顾持久化需求与离线可用性。
5. **UML 模型**：产出了包图、组件图、部署图、高层顺序图、详细类图、用例顺序图、活动图及状态图，从不同视角完整描述了系统的静态结构与动态行为。

5.2 设计优势与不足

设计优势：

- **全离线能力**：从文档编辑、格式转换到 RAG 问答，所有核心功能均可在无互联网环境下运行，完全本地化，保障学术数据隐私。
- **高扩展性**：AFD 模板体系通过 JSON 描述格式规范，用户可自定义期刊模板；PDF 转换采用策略链，可动态检测可用引擎；Lua 筛选器与 Normalizer 预处理链可独立扩展。
- **用户体验连贯**：双屏联动编辑与实时预览，配合高保真导出，形成“所见即所得”到“所得即所期”的闭环；RAG 问答直接嵌入写作界面，减少上下文切换。
- **鲁棒的错误处理**：每个核心用例均定义了详尽的异常场景与用户友好提示，转换管线异常通过 ConversionErrorFormatter 统一归集中文错误信息，降低用户困惑。

设计不足：

- **系统耦合于平台相关技术**：WebView2 运行时依赖 Windows 环境，虽可通过跨平台方案替换，但当前设计未完全屏蔽平台差异，迁移至 macOS/Linux 需重新设计编辑器宿主。

- **RAG 向量索引内存化**：当前嵌入向量全量缓存于内存，文献规模极大时 (>10 万页) 可能引起内存压力，设计未引入持久化向量数据库作为备选方案。
- **Pandoc 子进程管理**：对 Pandoc 子进程的依赖为强耦合，未抽象出通用文档转换接口，未来如需替换引擎需修改管线核心。
- **并发与多窗口支持缺失**：当前设计以单文档、单会话为假设，未考虑多文档同时转换、多标签编辑或同时打开多个工作区的需求。
- **部分设计决策未充分定量评估**：Reranker 模型选择、分块大小策略等依赖经验值，缺乏在目标文献集上的基准测试数据支

5.3 后续工作展望

架构解耦与跨平台演进：引入 Avalonia UI 或 .NET MAUI 替代 WPF + WebView2 绑定，同时抽象 PDF 查看器与编辑器宿主接口，使核心转换与 RAG 管线可复用至跨平台版本。

RAG 引擎强化：接入轻量级本地向量数据库（如 Qdrant 或 LanceDB），支持大规模文献库持久化索引；引入混合检索（关键词+向量）与多模态文献（图表、表格）处理能力。

协作与云增强：在保持离线能力的前提下，设计可选的同步服务，支持模板市场、写作协作与云端模型扩展（如按需下载 GGUF 模型）。

性能优化与测试：对超大文档 (>10 万字) 的实时预览与转换进行性能基准测试，优化 Markdown 增量解析算法；建立 AFD 样式映射自动化回归测试集。

用户反馈驱动的界面迭代：设计 A/B 测试与日志埋点方案，收集真实学者用户在格式导出准确率、问答满意度与编辑体验方面的数据，量化并持续优化设计。

文档转换引擎抽象：定义 `IDocumentConverter` 通用接口，使 PandocPipeline 成为一种可替换的实现，为未来支持更多源 / 目标格式（如 Typst、LaTeX）奠定架构基础。

参考文献

- [1] 《软件工程》课程讲义，2025 版.
- [2] IEEE Std 829-2008, *IEEE Standard for Software and System Test Documentation*, IEEE, 2008.
- [3] WeaveDoc 需求规格说明书，版本 V1.0.
- [4] Pandoc 用户手册，Lua 筛选器与 DOCX 输出配置. [Online]. Available: <https://pandoc.org/MANUAL.html>
- [5] Open XML SDK 文档，Microsoft. [Online]. Available: <https://learn.microsoft.com/en-us/office/open-xml/open-xml-sdk>
- [6] LLamaSharp 库文档，GGUF 模型加载与推理. [Online]. Available: <https://github.com/SciSharp/LLamaSharp>

[7] Monaco Editor API 文档, Microsoft. [Online]. Available: <https://microsoft.github.io/monaco-editor/>

[8] PDF.js 开发指南, Mozilla. [Online]. Available: <https://mozilla.github.io/pdf.js/>

[9] WebView2 开发者文档, Microsoft. [Online]. Available: <https://learn.microsoft.com/en-us/microsoft-edge/webview2/>

附录

附录 A 系统架构设计评审表

评审项目	评分标准（满分）	得分	评审意见
需求回顾与架构选型 (20分)	需求回顾全面，备选架构分析深入，选择理由充分且贴合项目（20）；较全面，分析较合理，理由较充分（15-19）；基本全面，分析基本合理（10-14）；不全面，分析不合理（0-9）	20	需求回顾涵盖了核与约束，十分全面微服务进行了合理架构的理由（离线源控制）充分且完
系统总体结构 (25分)	模块划分清晰，职责明确无重叠，分解逻辑合理（25）；较清晰，职责较明确（20-24）；基本清晰，职责基本明确（15-19）；不清晰，职责混乱（0-14）	25	划分为表示层、服献)、基础设施层、系统职责描述具体叠，符合分层架构
接口设计 (20分)	接口定义规范，参数、返回值明确，无歧义（20）；较规范，较明确（15-19）；基本规范，基本明确（10-14）；不规范，不明确（0-9）	18	子系统间接口（IEIRAGEngine、ICit接口（Pandoc CL定义清晰，用途明方法签名与返回值展开，但整体已较面补充主要方法的
关键技术与决策 (15分)	技术栈选择合理，决策记录完整，风险分析到位（15）；较合理，较完整，较到位（12-14）；基本合理，基本完整（9-11）；不合理，不完整（0-8）	15	技术栈（.NET 10、Avalonia UI、SQL分，重要决策（进导出、离线优先）存溢出、Pandoc冲突等风险均给出析到位。
UML 模型 (20分)	包图、组件图、顺序图完整准确，符合 PlantUML 规范，标注清晰（20）；较完整较准确，较规范（16-19）；基本完整基本准确（12-15）；不完整不准确（0-11）	17	架构层面规划了包及高层顺序图，结出图清单而未呈现位于附录，建议在引用附录。提供的（见下文）整体准足了设计的表达需
总分	100 分	95	
评审结论	☐ 通过 ☒ 修改后通过 ☐ 不通过		建议补充主要接口在正文中插入或明图。整体设计质量
评审人签字	王志钢	评审日期	2026-05-24

附录 B 界面设计评审表

评审项目	评分标准（满分）	得分	
用户分析与规范 (20 分)	用户分析深入，界面风格统一，设计规范详细（20）； 较深入，较统一，较详细（16-19）； 基本深入，基本统一（12-15）； 不深入，不统一（0-11）	19	已在 [范.md 使用 [格、字 计系统 距、颜
界面原型设计 (25 分)	原型完整覆盖所有主要界面，布局合理美观，标注清晰（25）； 较完整，较合理美观（20-24）； 基本完整，基本合理（15-19）； 不完整，不合理（0-14）	24	页面 [台、A 快照时 首次配 助/关 页面均 块、组
交互设计 (20 分)	交互流程合理，跳转关系明确，异常处理周全（20）； 较合理，较明确，较周全（16-19）； 基本合理，基本明确（12-15）； 不合理，不明确（0-11）	19	用户 [息架构 心任务 高频用 导出 [描 PD 恢复等
UML 模型 (20 分)	导航图、边界类图、顺序图完整准确，符合 PlantUML 规范（20）； 较完整较准确，较规范（16-19）； 基本完整基本准确（12-15）； 不完整不准确（0-11）	19	uml [图、边 图、部 出、 OCR、 图；

用户友好性 (15 分)	操作便捷，符合用户习惯，体验好（15）； 较便捷，较符合（12-14）； 基本便捷，基本符合（9-11）； 不便捷，不符合（0-8）	14	单窗口 核心路 状态栏 性快照 计符合 真实可 5
总分	100 分	95	文档作 构、 致；主 测试与 1
评审结论	☒ 通过 ☐ 修改后通过 ☐ 不通过	评审 人签 字	
评审日期	2026-05-24	-	

附录 C 详细设计评审表

评审项目	评分标准（满分）	得分	评审意见
用例详细设计 (20 分)	用例细化全面，步骤清晰，前置 / 后置条件明确，异常处理完整（20）； 较全面，较清晰，较完整（16-19）； 基本全面，基本清晰（12-15）； 不全面，不清晰（0-11）	18	详细设计中选取了“学术文档 RAG 文献问答”“双屏联动编辑用例，基本覆盖了系统最重要用例均给出了参与者、前置条件步骤和异常处理，流程较完整可以进一步补充用户取消操作边界情况。
类设计 (25 分)	类识别完整，属性方法定义准确，关系清晰，算法逻辑明确（25）； 较完整，较准确，较清晰（20-24）； 基本完整，基本准确（15-19）； 不完整，不准确（0-14）	23	类设计按照表示层、编辑器子系统、AI 辅助子系统、AFD 系统进行了划分，核心类识别较完整。 <code>DocumentConversionEngine</code> 、 <code>LocalAiService</code> 、 <code>Pandora</code> 类给出了职责、核心方法和外部组合、依赖和策略模式关系也持续进一步统一部分类名与接口职责过重。
数据设计 (20 分)	E-R 图完整准确，表结构设计合理，约束明确（20）； 较完整较准确，较合理（16-19）； 基本完整基本准确（12-15）； 不完整不准确（0-11）	18	数据设计说明了 SQLite 数据库缓存三类存储方式，其中 term 键和约束定义较清楚，能够支持。文件系统结构也列出了模型文件等内容。RAG 向量索引符合本地优先和轻量化设计目标。E-R 图中的实体关系说明，并效时的数据一致性处理。
UML 模型 (25 分)	详细类图、顺序图、活动图等完整准确，符合 PlantUML 规范（25）； 较完整较准确，较规范（20-24）； 基本完整基本准确（15-19）； 不完整不准确（0-14）	22	已规划并绘制详细类图、三个顺序图、关键用例活动图、状态类型较完整，能够从不同角度设计。PlantUML 文件命名较清晰间基本对应。
可追溯性 (10 分)	设计与需求对应关系明确，每个决策有依据（10）； 较明确，较有依据（8-9）； 基本明确，基本有依据（6-7）； 不明确，无依据（0-5）	9	详细设计能够对应前期需求中档案导出、RAG 文献问答、双屏选择与 MoSCoW 优先级基本对应系统架构中的 MVVM、AI 辅助子系统和本地优先设计。增加“需求编号—用例—设计表，使可追溯性更加直观。
总分	100 分	90	

评审结论	☒ 通过 ☐ 修改后通过 ☐ 不通过	评审 人签 字	任逸青
评审日期	2026-05-24		



附录 D PlantUML 源代码

见附件相关文件夹下"src"文件夹

附录 E UML 图 PNG 图片

见附件相关文件夹下"img"文件夹